

Digital Electronics

for VLSI and Embedded Systems



FRANKLIN OSES J

Advanced Design & Implementation

About the Author

Franklin Moses is a passionate engineer specializing in Digital Electronics, VLSI Design, and Embedded Systems. With a strong foundation in digital logic, CMOS fundamentals, and semiconductor device concepts, he has developed multiple innovative projects in the areas of image-processing-based automation, IoT-enabled power systems, and hardware-implemented digital design.

His work includes the development of an Automatic Waste Segregation System using image processing, IoT-based Transmission Line Fault Detection, and dual-axis solar tracking hardware using microcontrollers. His innovative solutions earned him several recognitions, including the MSME State-Level Runner-Up Award, with media coverage in Dinakaran under Manavan Minital.

Franklin has hands-on experience with industry-standard VLSI tools, including Synopsys Design Compiler, IC Compiler II, Fusion Compiler, and Star-RC, along with practical exposure to Arduino, ESP8266, and Raspberry Pi platforms. His technical interests span RTL-to-GDSII flow, physical design optimization, timing closure, and low-power digital architecture.

Franklin aims to contribute to advanced chip design and deep-tech innovation, focusing on ASIC and SoC development. This book reflects his vision to simplify complex digital design concepts and make VLSI fundamentals accessible to students, aspiring engineers, and young researchers.

TABLE OF CONTENT

CHAPTER 1 – Number Systems

CHAPTER 2 – Number System Conversions

CHAPTER 3 – Binary Arithmetic Operations

CHAPTER 4 – Logic Gates and Universal Gates

CHAPTER 5 – Boolean Algebra and Boolean Functions

CHAPTER 6 – Canonical and Standard Forms

CHAPTER 7 – Karnaugh Map (K-Map) Simplification

CHAPTER 8 – Combinational Logic Circuits

CHAPTER 9 – Adders

CHAPTER 10 – Multiplexers (MUX)

CHAPTER 11 – Demultiplexers (DEMUX)

CHAPTER 12 – Encoders and Decoders

CHAPTER 13 – Comparators

CHAPTER 14 – Multipliers

CHAPTER 15 – Sequential Logic Circuits

CHAPTER 16 – Latches

CHAPTER 17 – Flip-Flops

CHAPTER 18 – Registers and Counters

CHAPTER 19 – Finite State Machines (FSM)

CHAPTER 1 – NUMBER SYSTEMS

1.1 Introduction

In digital electronics and VLSI design, all information is represented using numbers. Whether it is data stored in a register, an address in memory, a logic value on a wire, or switching activity inside a flip-flop, everything is handled in **numerical form**. To process this information efficiently, digital systems use number systems, which define how values are written and interpreted inside hardware.

While humans commonly use the decimal system, digital circuits use the **binary system** because it directly matches the two stable voltage levels in CMOS technology:

- 0 → LOW (GND)
- 1 → HIGH (VDD)

Other number systems such as octal and hexadecimal are also used because they provide compact, readable representations of long binary values. Hexadecimal, in particular, is widely used in VLSI for:

- Memory addresses
- Register maps
- Instruction codes
- Debugging and waveform analysis
- Physical design coordinates (DEF/GDS)

Understanding number systems is essential in VLSI because it helps engineers translate data values, interpret simulation results, understand instruction formats, and design arithmetic circuits like adders and multipliers.

In this chapter, we explore the different number systems, their conversions, and basic arithmetic operations, forming the foundation for all higher-level digital and physical design concepts.

1.2 Types of Number Systems

Digital systems use different number systems to represent and process information. Each number system has a **base (radix)** and a set of **allowed digits**. In VLSI and digital electronics, four number systems are commonly used: **Binary, Octal, Decimal, and Hexadecimal**. Each serves a different purpose depending on how data needs to be represented or interpreted.

Computer architecture supports following number systems.

NUMBER SYSTEM	BASE(RADIX)	USED DIGITS	EXAMPLE
Binary	2	0,1	(11110000) ₂
Octal	8	0,1,2,3,4,5,6,7	(360) ₈
Decimal	10	0,1,2,3,4,5,6,7,8,9	(240) ₁₀
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9, A,B,C,D,E,F	(F0) ₁₆

1.2.1 Binary Number System

Digits used:0,1

Radix: 2

Binary is the **primary number system** used in digital electronics because it matches the two stable states of CMOS circuits:

- 0 → Low voltage (GND)
- 1 → High voltage (VDD)

Every hardware element—logic gates, flip-flops, registers, SRAM cells—stores and processes information in binary. All operations in VLSI (addition, subtraction, shifting, memory addressing, instruction execution) are performed using binary values.

Example: 1011₂ represents a binary number.

1.2.2 Octal Number System

Digits used:0–7

Radix: 8

The octal number system is used as a shorter way to represent binary numbers because one octal digit corresponds to three binary bits. It makes long binary values easier to read and handle.

- 1 octal digit = 3 binary bits
- Useful for grouping and simplifying binary data

Example: 47₈ represents an octal number.

1.2.3 Decimal Number System

Digits used: 0–9

Radix: 10

The decimal system is the number system humans commonly use in daily life. Although digital circuits do not operate directly on decimal values, decimal numbers are still used for input values, timing parameters, and specifications.

- Human-friendly representation
- Must be converted to binary before hardware processing

Example: 92_{10} represents a decimal number.

1.2.4 Hexadecimal Number System

Digits used: 0–9 and A–F

Radix: 16

The hexadecimal number system is widely used in digital and VLSI design because it represents large binary numbers in a compact, readable form. One hexadecimal digit is equal to four binary bits.

- 1 hex digit = 4 binary bits
- Used in memory addresses, register maps, and debugging

Example: AF_{16} represents a hexadecimal number

1.3 Use of Number System

➤ Binary System — Usage

Used by computers for all operations and data processing.

Example: Computer language (0 and 1 signals).

➤ Octal System — Usage

Used for file permissions and shortcuts for binary.

Example: UNIX permissions like **755**.

➤ Decimal System — Usage

Used for everyday counting and calculations.

Example: Using a **calculator** to add numbers.

➤ Hexadecimal System — Usage

Used for colour codes, memory addresses, and machine instructions.

Example: Colour code **#FF00AA**.

CHAPTER 2 – NUMBER SYSTEM CONVERSIONS

2.1 Introduction to Conversions

Number system conversions help us change a number from one base to another. Even though we use decimal in daily life, **computers use binary**. So conversions help humans understand what the computer is doing.

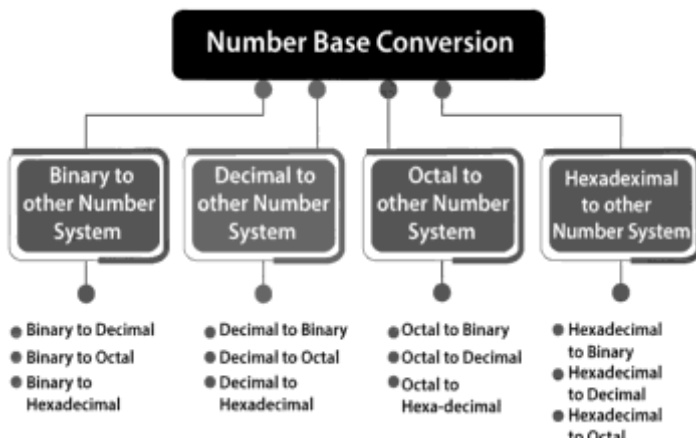
Number system conversions are used everywhere around us — in phones, computers, calculators, QR scanners, and even screens. They help convert **machine-friendly** binary values into **human-friendly** decimal or hex formats. Without these conversions, we would not be able to understand or interact with any digital device easily. So, learning conversions is important for both everyday technology use and for engineering applications.

Here are simple, real, practical usages:

1. Mobile Phone Storage (Binary → Decimal): Your phone says 64 GB, but internally it counts memory in binary ($1024 \text{ MB} = 1 \text{ GB}$). Because humans understand decimal better, the phone converts binary values to decimal for display. This conversion helps show storage in a simple way instead of long binary numbers.

2. Calculator Operations (Decimal ↔ Binary): When you press $8 + 2 = 10$, you enter numbers in decimal. Inside the calculator, everything is converted to binary for calculation. After processing, it converts the result back to decimal so you can read it easily.

3. Screen Colour Codes (Hex → Decimal): Colours like #FF0000 are written in hexadecimal on apps and design tools. Your phone converts them into decimal RGB values like (255, 0, 0).



2.2 Binary to Octal Conversion

Group binary digits into **3 bits from right** and convert each group to octal. **Example:** 101 110 → 5 6 → **56₈**

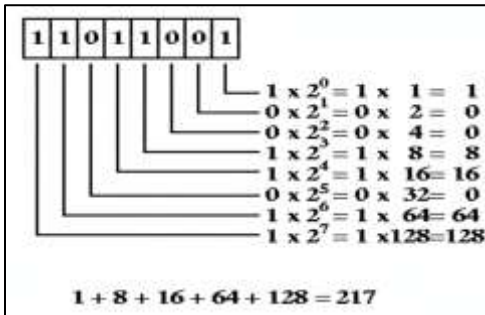
Binary Number:	101010011.110100
Group of three digits:	101 010 011. 110 100
Octal Equivalent:	5 2 3 6 4
	= (523.64) ₈

Used in digital circuits — octal groups reduce long binary strings, making it easier for engineers to read hardware data.

2.3 Binary to Decimal Conversion

Multiply each bit by powers of 2 and add.

Example: 1011₂ = 1×8 + 0×4 + 1×2 + 1×1 = **11₁₀**

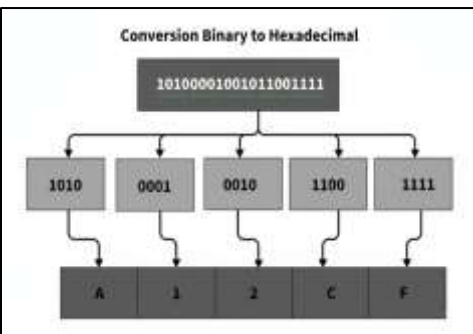


Used in calculators and digital meters — you enter decimal values, but the device converts binary results back into decimal to display.

2.4 Binary to Hexadecimal Conversion

Group binary digits into **4 bits from right** and convert each group to hex.

Example: 1101 0110 → D6₁₆

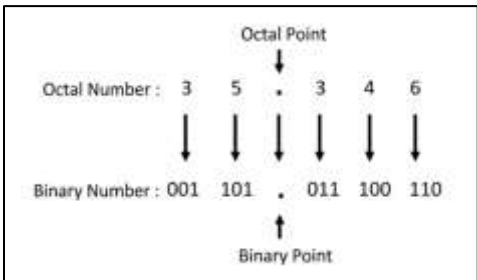


Used in Wi-Fi MAC addresses and IP packets — long binary data is shortened into readable hex format.

2.5 Octal to Binary Conversion

Convert each octal digit into **3-bit binary**.

Example: $7 \rightarrow 111, 4 \rightarrow 100 \rightarrow 111100_2$



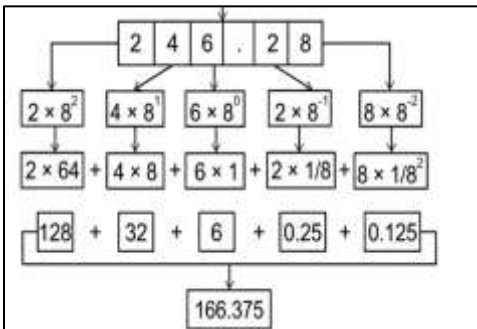
Used in old operating systems and file permission systems

(e.g., Linux 755 permissions convert to binary).

2.6 Octal to Decimal Conversion

Multiply each digit by powers of 8.

Example: $732_8 = 7 \times 64 + 3 \times 8 + 2 = 474_{10}$

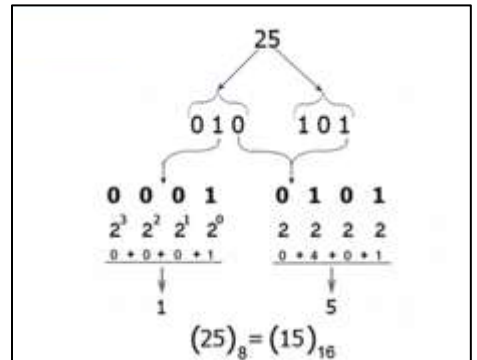


Used when converting old machine instruction codes into decimal for documentation or hardware upgrades.

2.7 Octal to Hexadecimal Conversion

Octal $\rightarrow$ Binary $\rightarrow$ Hex

Example: $157_8 \rightarrow 001\ 101\ 111 \rightarrow 1DF_{16}$



Used when converting old machine instruction codes into decimal for documentation or hardware upgrades.

2.8 Decimal to Binary Conversion

Repeated division by 2 (or subtraction of powers of 2).

Example: $25 \rightarrow 11001_2$

(10.625)₁₀

2	10	
2	5	0
2	2	1
2	1	0
	0	1

Decimal to Binary

0.625

$0.625 \times 2 = 1.250$	-----	1
$0.250 \times 2 = 0.500$	-----	0
$0.500 \times 2 = 1.0$	-----	1
0×2		

(16.625)₁₀ = (1010.101)₂

Used in microcontrollers — decimal values you enter (like pin numbers) are converted to binary for hardware control.

2.9 Decimal to Octal Conversion

Repeated division by 8.

Example: $65 \rightarrow 101_8$

675

16	675	
16	42	3
	2	10

(2 10 3)
(2 A 3)
(675)₁₀ = (2A3)₁₆

Used in microcontrollers — decimal values you enter (like pin numbers) are converted to binary for hardware control.

2.10 Decimal to Hexadecimal Conversion

Repeated division by 16.

Example: $254 \rightarrow FE_{16}$

29.30

8	29	
8	3	5
	0	3

$0.30 \times 8 = 2.4$

$0.40 \times 8 = 3.2$

$0.20 \times 8 = 1.6$

$0.60 \times 8 = 4.8$

$0.80 \times 8 = 6.4$

$0.40 \times 8 = 3.2$ Stop

$0.20 \times 8 = 1.6$ Repeated

$0.60 \times 8 = 4.8$

0.80

(29.30)₁₀ = (35.23146)₈

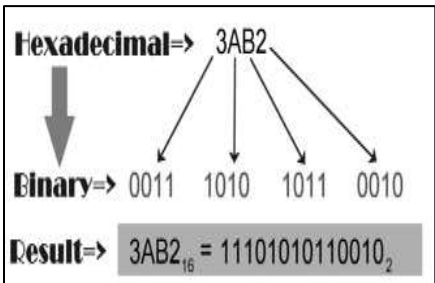
circuitspedia.com

Used in screen color codes — RGB decimal values convert to hex like **#FF8800** for apps and websites.

2.11 Hexadecimal to Binary Conversion

Convert each hex digit into **4-bit binary**.

Example: $A3_{16} \rightarrow 1010\ 0011_2$

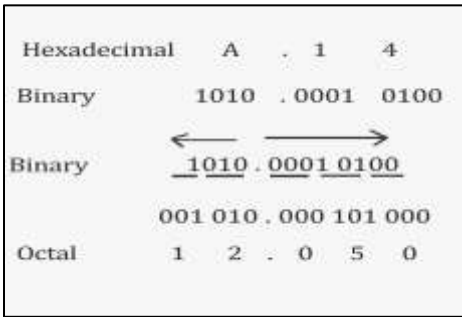


Used in debugging — hex memory addresses convert to binary to inspect individual bits.

2.12 Hexadecimal to Octal Conversion

Hex $\rightarrow$ Binary $\rightarrow$ Octal

Example: $9E_{16} \rightarrow 1001\ 1110 \rightarrow 236_8$

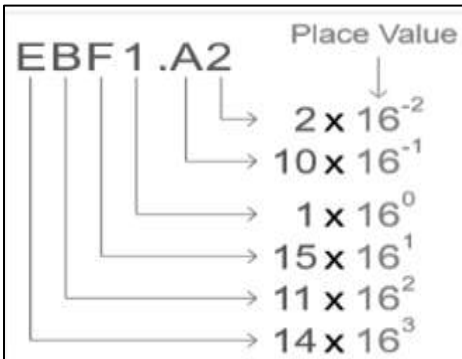


Used in compilers — hex memory locations are shown in decimal when analyzing system performance.

2.13 Hexadecimal to Decimal Conversion

Multiply each digit by powers of 16.

Example: $2F_{16} = 2 \times 16 + 15 = 47_{10}$



Used in industrial controllers — hex register values convert to octal for older machine interfaces.

CHAPTER 3 – BINARY ARITHMETIC OPERATIONS

3.1 Introduction

Binary arithmetic deals with performing mathematical operations using only two digits: 0 and 1. All digital computers, processors, and logic circuits use binary arithmetic internally. Understanding binary operations helps in learning how computers add, subtract, multiply, and divide data.

3.2 Binary Addition

It is a key for binary subtraction, multiplication, division. There are four rules of binary addition.

Case	A + B	Sum	Carry
1	0 + 0	0	0
2	0 + 1	1	0
3	1 + 0	1	0
4	1 + 1	0	1

Four rules of binary

EXAMPLE 1:

1010

1110

11000

EXAMPLE 2:

1001011

0100110

1110001

3.3 Binary Subtraction

Binary subtraction follows four basic rules: $0-0=0$, $1-0=1$, $1-1=0$, and $0-1$ requires borrowing from the next higher bit. When a borrow occurs, the 0 becomes 2 in binary (i.e., 10_2), making the subtraction possible. Binary borrowing is similar to decimal borrowing but works with base-2 values.

Input A	Input B	Subtract (S) A-B	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

EXAMPLE 1:

SUBTRACT $(1010)_2 - (0011)_2$

1 0 1 0
- 0 0 1 1

0 1 1 1

EXAMPLE 2:

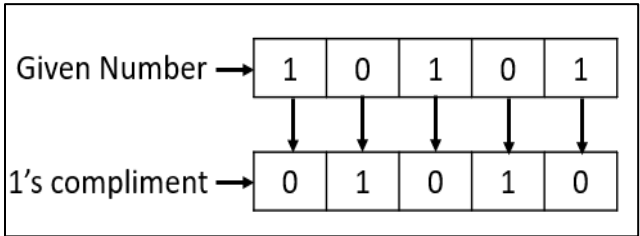
SUBTRACT $(1101)_2 - (1011)_2$

1 1 0 1
- 1 0 1 1

0 0 1 0

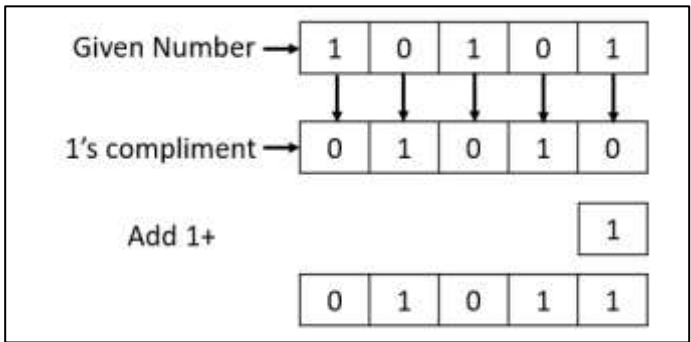
3.4 1's Complement Method

1's complement is obtained by **flipping all bits** ($0 \rightarrow 1$, $1 \rightarrow 0$). Subtraction using 1's complement is done by adding the complement of the subtrahend. If a carry is generated, it is added back (end-around carry).



3.5 2's Complement Method

2's complement is obtained by **taking 1's complement + 1**. It is the most widely used method for binary subtraction in computers. The result directly represents positive and negative numbers without extra steps.



3.6 Binary Multiplication

Binary multiplication is a method of multiplying two numbers using only 0 and 1. It works like normal multiplication: each bit is multiplied, and when the bit is 1, the other number is written as a partial product; when it is 0, the result is 0. Each partial product is shifted left based on its position, and all partial products are then added using binary addition. This method is used inside processors and digital circuits for arithmetic operations.

Input A	Input B	Multiply (M) AxB
0	0	0
0	1	0
1	0	0
1	1	1

EXAMPLE 1:

1010

x 101

1010

0000*

+ 1010**

110010

(1010)₂ x (101)₂ = (110010)₂

EXAMPLE 2:

101.01

x 110

00000

10101*

+ 10101**

1111110

= 11111.10

(101.01)₂ x (110)₂ = (11111.10)₂

3.7 Binary Division

Binary division is similar to decimal division but uses only 0s and 1s. It follows the same process: compare the divisor with the dividend, subtract when possible, and bring down the next bit. If the divisor fits, write 1 in the quotient; if not, write 0. This process continues until all bits are processed, giving the quotient and remainder in binary.

EXAMPLE 1:

111

110

101101

- 110

1010

- 110

1001

- 110

11

$(101101)_2 \div (110)_2 = (111)_2$

EXAMPLE 2:

10100.1110

100

1010011.1010

- 100

100

- 100

111

- 100

110

- 100

101

- 100

10

$(1010011.1010)_2 \div (100)_2 = (10100.110)_2$

3.8 Applications of Arithmetic Operations in VLSI

Basic Calculations – Chips need arithmetic to do simple math like add, subtract, multiply, and divide.

Processor Working – The CPU uses arithmetic to run instructions and calculate addresses.

Signal Processing – Audio, video, and sensor data need fast math operations inside hardware.

Memory Access – To read/write data, the chip calculates memory locations using arithmetic.

Communication – Arithmetic helps check and correct errors in data transfer between devices.

Graphics & Gaming – GPUs use arithmetic to calculate colors, shapes, and animations.

Machine Learning – AI hardware performs huge numbers of multiplications and additions.

Control Systems – Devices like motors and robots make decisions using arithmetic formulas.

Cryptography – Secure chips use math to encrypt and protect data.

Performance & Speed – Doing arithmetic in hardware is faster and uses less power than software.

Timers and Counters – Arithmetic is used to count clock cycles, measure time, and generate delays.

Sensor Calibration – Chips use arithmetic to convert raw sensor values into real-world units (temperature, pressure, etc.).

Data Compression – Video/audio compression uses math operations to reduce file size (JPEG, MP3, H.264).

Power Management – Arithmetic helps calculate power usage, battery level, and manage voltage scaling.

Robotics and Automation – Robots use arithmetic for motion control, path planning, and distance calculation.

CHAPTER 4 – LOGIC GATES AND UNIVERSAL GATES

4.1 Introduction to Digital Logic

Digital logic is a fundamental area of electronics that uses binary values (0 and 1) to process information. Unlike analog circuits that work with continuous signals, digital logic circuits operate using discrete voltage levels to represent logical states such as LOW/HIGH, OFF/ON, or FALSE/TRUE. This makes digital systems highly reliable, noise-resistant, and easier to design for complex operations. Digital logic is based on Boolean algebra, where mathematical operations are performed using logical relations. These operations are implemented using logic gates, such as AND, OR, NOT, NAND, NOR, XOR, and XNOR. By combining these basic logic gates, we can create larger functional blocks called combinational circuits (like adders, multiplexers) and sequential circuits (like flip-flops, counters, registers). Digital logic forms the backbone of all modern computing and electronic systems. Every processor, memory device, and communication IC is built using digital logic concepts. It enables the design of VLSI (Very Large Scale Integration) chips, where millions of logic gates are integrated onto a single silicon chip to perform complex operations at high speed.

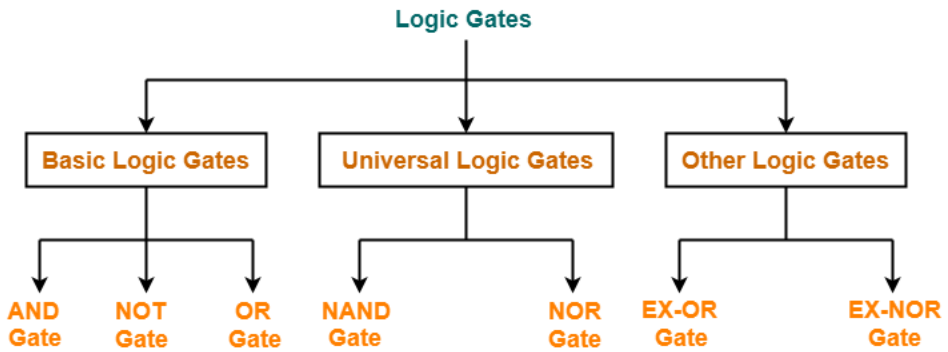
4.2 BASIC LOGIC BLOCKS

Binary Digital Input Signal → Gate → Binary Digital Output Signal



Types of Basic Logic Blocks

- **Combinational Logic Block:** Logic blocks whose output logic value depends only on the input logic values.
- **Sequential Logic Block:** Logic blocks whose output logic value depends on the input values and the state (stored information) of the blocks.

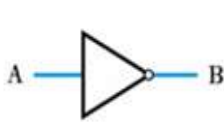


Types of Logic Gates

4.3 Basic Gates

➤ Inverter/Not Gate:

A NOT gate is a basic digital logic gate that inverts the input signal. It produces an output that is the logical opposite of its input. If the input is 1, the output will be 0; and if the input is 0, the output will be 1. Because it reverses the input logic level, it is also called an Inverter.



(a) Circuit symbol

A	B
0	1
1	0

(b) Truth table

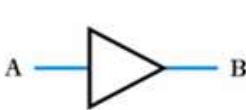
$$B = \bar{A}$$

(c) Boolean expression

Inverter/NOT Gate

➤ Buffer Gate:

A Buffer gate is a digital logic gate that transfers the input signal directly to the output without any change. It provides an output that is exactly the same as the input (0 gives 0, 1 gives 1). Buffers are mainly used to increase signal strength, drive larger loads, and prevent signal distortion in digital circuits.



(a) Circuit symbol

A	B
0	0
1	1

(b) Truth table

$$B = A$$

(c) Boolean expression

Buffer Gate

➤ **AND Gate:**

An AND gate is a basic digital logic gate that gives an output of 1 only when all its inputs are 1. If any input is 0, the output will be 0. It performs the logical AND operation based on Boolean algebra.



(a) Circuit symbol

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

$C = A \cdot B$

(c) Boolean expression

AND Gate

➤ **NAND Gate:**

A NAND gate is a digital logic gate that gives an output of 0 only when all its inputs are 1. In all other input conditions, the output is 1. It performs the NOT-AND operation, which is the inverted result of an AND gate.



(a) Circuit symbol

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

(b) Truth table

$C = \overline{A \cdot B}$

(c) Boolean expression

NAND Gate

➤ **OR Gate:**

An OR gate is a basic digital logic gate that gives an output of 1 if any one of its inputs is 1. The output will be 0 only when all inputs are 0. It performs the logical OR operation based on Boolean algebra.



(a) Circuit symbol

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

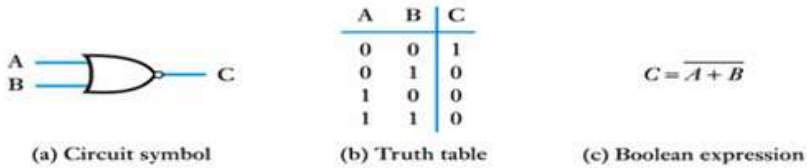
$C = A + B$

(c) Boolean expression

OR Gate

➤ **NOR Gate:**

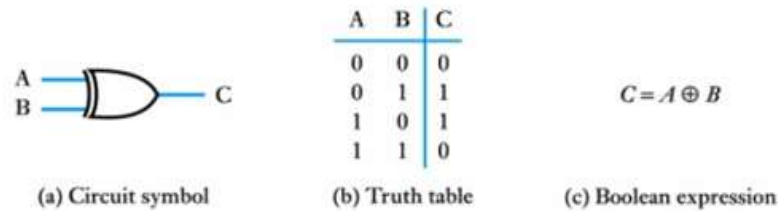
A NOR gate is a digital logic gate that gives an output of 1 only when all its inputs are 0. If any input is 1, the output becomes 0. It performs the NOT-OR operation, which is the inverted result of an OR gate.



NOR Gate

➤ **EX-OR Gate:**

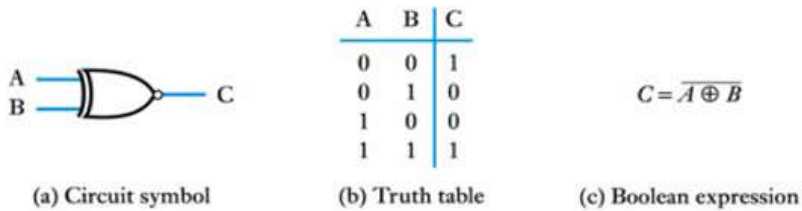
An EX-OR (Exclusive-OR) gate is a digital logic gate that gives an output of 1 when the inputs are different. If both inputs are the same (0-0 or 1-1), the output will be 0. It performs the exclusive OR operation, which detects difference between inputs.



EX-OR Gate

➤ **EX-NOR Gate:**

An EX-NOR (Exclusive-NOR) gate is a digital logic gate that gives an output of 1 when the inputs are the same. If the inputs are different, the output will be 0. It performs the inverse of the XOR operation, detecting similarity between inputs.



EX-NOR Gate

4.4 Universal Gates

Universal gates are special logic gates that can be used to build any Boolean logic function. By combining only these gates, we can implement all other basic logic gates such as AND, OR, NOT, XOR, etc. The two universal gates are:

1. NAND GATE
2. NOR GATE

They are called **universal** because:

- We can build **complete digital circuits** using only **one type** of gate.
- They reduce the need for multiple gate types in hardware design.
- They allow **optimized and simplified circuit design** in VLSI.

1. NAND Gate

A NAND gate can be connected in different ways to create:

- NOT gate
- AND gate
- OR gate
- NOR gate
- XOR and XNOR (with combinations)

Because of this flexibility, NAND gates are widely used in digital circuits and VLSI design.

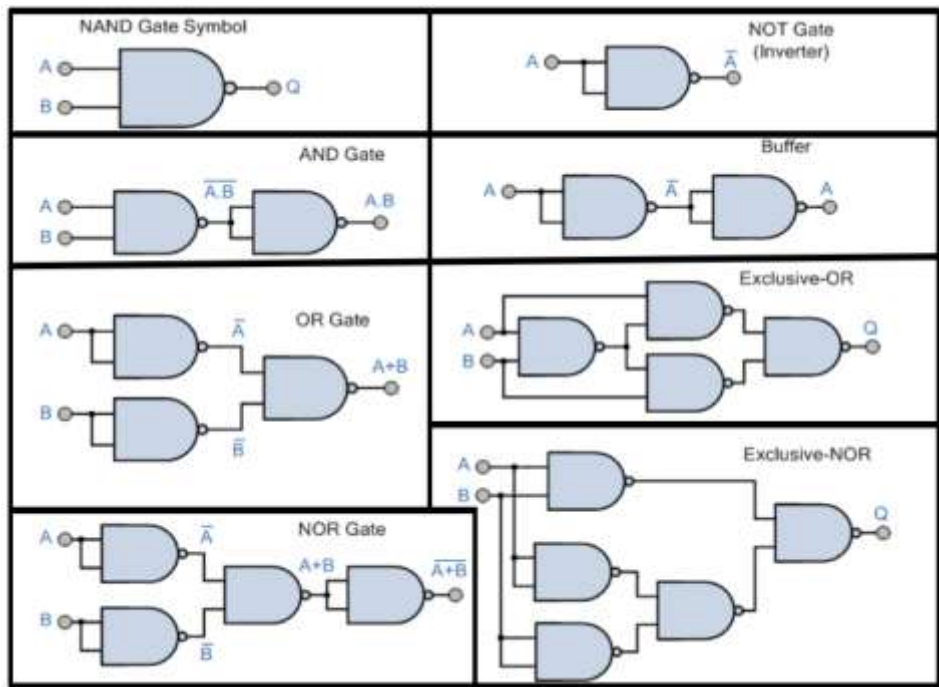
2. NOR Gate

A NOR gate can also be used to implement all logic functions, including:

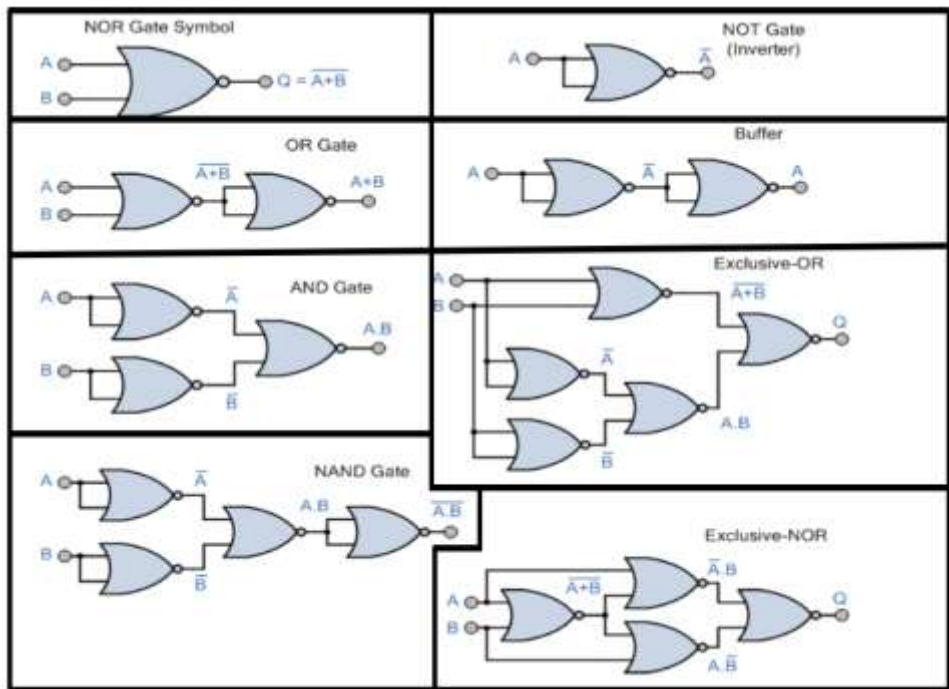
- NOT gate
- OR gate
- AND gate
- NAND gate
- XOR and XNOR (with combinations)

NOR gates are commonly used in **sequential circuits**, such as **flip-flops**, because of their feedback properties.

4.5 Logic Gates Implementation Using NNAD Gates



4.6 Logic Gates Implementation Using NOR Gates



CHAPTER 5 – BOOLEAN ALGEBRA AND BOOLEAN FUNCTIONS

5.1 Boolean Variables and Expressions

Boolean algebra is a mathematical system used to describe and analyze digital logic circuits. It works with variables that can have only two possible values, usually represented as 0 and 1, corresponding to LOW/HIGH, FALSE/TRUE, or OFF/ON states in digital electronics.

A Boolean variable represents a quantity that can take only these two values. Example: $A = 0$ or $A = 1$. In a logic circuit, variables are used as inputs and outputs of logic gates.

A Boolean expression is a mathematical formula written using Boolean variables, logical operators, and constants (0 and 1). The most commonly used logical operators are:

- AND ($\cdot$)
- OR ($+$)
- NOT ($'$)

These operators follow the rules of Boolean algebra and define the relationship between input variables to produce a desired output.

Boolean expressions can be represented using:

- Boolean equations
- Truth tables
- Logic gate diagrams

For example, the expression

$$Y = A \cdot B + \bar{C}$$

means that the output Y is TRUE (1) when A and B are both 1, or when C is 0.

Boolean algebra is essential for simplifying logic circuits, minimizing the number of gates, and designing efficient combinational and sequential circuits in digital systems.

5.2 Boolean Algebra Laws

Boolean Algebra is a mathematical system used to analyze and simplify logic circuits. It works with binary variables that take only two values: 0 (LOW) or 1 (HIGH). The operations are mainly AND ($\cdot$), OR ($+$), and NOT ($\bar{}$). To simplify Boolean expressions, a set of basic algebraic laws are used. These laws help reduce logic functions to their simplest form.

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{\bar{A} + \bar{B}} = A\bar{B}$

Conclusion

These Boolean algebra laws are essential for simplifying logic expressions and designing digital circuits. By applying these rules, complex logical equations can be reduced, making circuit implementation easier and cost-effective.

5.3 Simplification Techniques

In digital logic design, Boolean expressions are often complex and contain many terms. Simplification is the process of reducing these expressions into their simplest form without changing the output. The simplified form results in fewer logic gates, reduced cost, faster operation, and increased reliability of digital circuits.

Several techniques are used for simplifying Boolean functions. The most common methods are:

5.3.1 Boolean Algebra Simplification

Boolean Algebra uses a set of algebraic laws and theorems to reduce expressions step-by-step. By applying laws such as **Identity, Null, Idempotent, Complement, Associative, Commutative, Distributive, Absorption, and De-Morgan's Theorem**, the expression can be simplified.

Steps:

1. Identify common terms.
2. Apply absorption and distributive laws.
3. Use inverse (complement) and idempotent laws.
4. Remove redundant terms.
5. Use De-Morgan's theorem if required.

Example:

Simplify:

$$Y = A + AB$$

Using Absorption Law,

$$Y = A$$

So, the expression is reduced to a single term.

5.3.2 Karnaugh Map (K-Map) Simplification

Karnaugh Map is a graphical method used to simplify Boolean functions of **2, 3, and 4 variables**. It is an organized way of grouping adjacent 1s in a truth table.

Advantages:

- Provides a visual simplification approach.
- Ensures minimum expression.
- Reduces possibilities of mistakes.
- Useful for small and medium-size functions.

Steps:

1. Draw the K-Map based on the number of variables.
2. Enter the output values (0 or 1) from the truth table.
3. Group adjacent 1s into blocks (groups of 1, 2, 4, 8).
4. Write the simplified expression from the groups.

5.3.3 Algebraic Factorization

Sometimes Boolean functions can be simplified by factoring common terms, similar to normal algebra.

Example:

$$F = AB + AC$$

Factor A:

$$F = A(B + C)$$

So, two terms reduce to one AND gate and one OR gate.

5.3.4 Using De-Morgan's Theorem

De-Morgan's theorem is helpful when dealing with NAND/NOR implementations or when the expression contains complemented groups.

Theorem:

- $(A + B)' = A'B'$
- $(AB)' = A' + B'$

Example:

Convert into sum-of-products:

$$F = (AB)'$$

$$F = A' + B'$$

5.3.5 Sum of Products (SOP) and Product of Sums (POS)

SOP and POS are canonical forms of Boolean expressions.

SOP

Expression is a sum (OR) of product (AND) terms.

Example:

$$F = AB + \bar{A}C + BC$$

SOP is easy to simplify using K-MAP.

POS

Expression is a product (AND) of sum (OR) terms.

Example:

$$F = (A + B)(\bar{A} + C)$$

POS can be simplified by expanding and reducing.

5.3.6 NAND and NOR Based Simplification

In practical digital circuits, NAND and NOR gates are universal gates. Simplification can convert expressions into NAND-only or NOR-only forms for cost-effective design.

Example:

$$F = (A + B)'$$

This directly represents a **NOR gate** output.

5.3.7 Consensus Theorem

The consensus theorem is used to remove redundant terms from Boolean expressions.

Theorem:

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

The term **BC** is redundant.

Benefits of Simplification

- Reduces number of logic gates.
- Minimizes cost and circuit size.
- Reduces propagation delay.
- Improves system reliability.
- Saves power consumption.

Conclusion

Simplification techniques are essential in digital logic design. Depending on the complexity and number of variables, Boolean algebra, K-Maps, SOP/POS methods, and De-Morgan's theorem are used to reduce the logic expression to its simplest form. Simplified Boolean functions allow efficient implementation of digital circuits.

5.5 Real Examples of Boolean Simplification

Boolean simplification is used in real digital circuits such as arithmetic units, multiplexers, control logic, and embedded systems. By simplifying Boolean expressions, the number of logic gates is reduced, which minimizes the cost, size, and power consumption of digital hardware.

The following examples illustrate the practical use of Boolean identities in simplifying logic expressions.

Example 1: Control Signal Logic

A system generates a control signal defined by: $F = A + AB$

Solution

Using Absorption Law: $A + AB = A$

Explanation

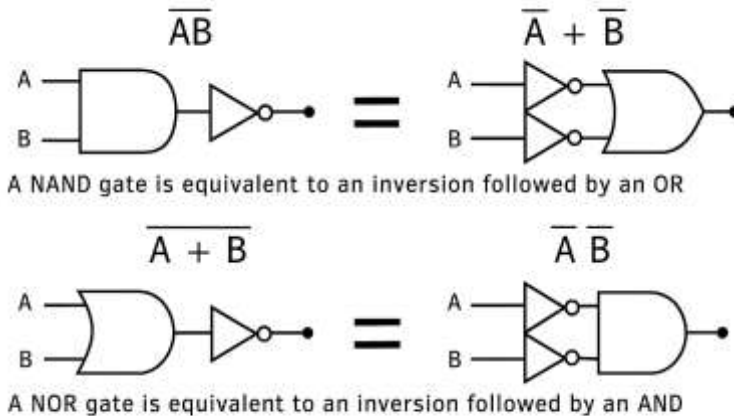
If A is 1, the output is always 1 regardless of B. So, the AND term **AB** is unnecessary. Only one input (A) is required.

Reduced circuit uses one OR gate less.

5.6 De Morgan's Theorem

De Morgan's Theorem is one of the most important concepts in Boolean Algebra. It shows the relationship between the **AND**, **OR**, and **NOT** operations. The theorem provides a method to simplify complemented Boolean expressions and is widely used in digital logic design, especially when working with NAND and NOR gates.

De Morgan's Theorem consists of **two fundamental laws**:



De Morgan's Theorem is Important

1. **Gate Conversion:** It helps convert AND to OR and OR to AND using NOT operation. Useful for implementing logic using **NAND** or **NOR** only.
2. **Simplification:** Complex expressions with complemented terms can be simplified.
3. **Digital Circuit Design:** In VLSI and FPGA design, De Morgan's theorem reduces gate count.
4. **Complementing a Function:** When finding the complement of a Boolean function, De Morgan's rules are applied.

Conclusion: De Morgan's Theorem establishes a direct link between AND, OR, and NOT operations. It states that the complement of an OR function is equal to the AND of complements, and the complement of an AND function is equal to the OR of complements. The theorem plays a vital role in simplifying Boolean expressions and designing efficient digital circuits.

CHAPTER 6 – CANONICAL AND STANDARD FORMS

Canonical and standard forms are systematic ways of expressing Boolean functions using logical operations. These forms help in the simplification and implementation of Boolean functions in digital circuits.

Canonical forms represent Boolean expressions using all variables in each term, whereas standard forms may not use all variables in each product or sum. Canonical expressions can be directly obtained from the truth table of a function.

6.1 Minterms and Maxterms

Boolean functions can be expressed in two canonical forms based on minterms and maxterms.

Minterm

A minterm is a product (AND) term that contains all variables, either in complemented or un-complemented form. A minterm is 1 for exactly one combination of input variables.

For two variables A and B, the minterms are:

A	B	Minterm
0	0	$\bar{A}\bar{B}$
0	1	$\bar{A}B$
1	0	$A\bar{B}$
1	1	AB

A minterm is represented as m_i , where i is the decimal value of binary inputs.

Example:

For $A = 1, B = 0 \rightarrow$ binary = 10 $\rightarrow$ decimal = 2 $\rightarrow$ minterm = $m_2 = A\bar{B}$

Maxterm

A maxterm is a sum (OR) term that contains all variables, either in complemented or un-complemented form. A maxterm is 0 for exactly one combination of input variables.

For two variables A and B, the maxterms are:

A	B	Maxterm
0	0	$A + B$
0	1	$A + \bar{B}$
1	0	$\bar{A} + B$
1	1	$\bar{A} + \bar{B}$

A maxterm is represented as M_i , where i is the decimal value of the binary inputs.

Example:

For $A = 1, B = 1 \rightarrow$ binary = 11 $\rightarrow$ decimal = 3 $\rightarrow$ maxterm = $M_3 = \bar{A} + \bar{B}$

6.2 Sum of Products (SOP)

The Sum of Products form expresses a Boolean function as a sum (OR) of product (AND) terms. Each product term may contain one or more variables.

General Form

$$F = A B + \bar{A} C + B \bar{C}$$

In SOP, each product term represents a condition when the output is 1.

Canonical SOP (Sum of Minterms)

In canonical SOP, every product term is a minterm, meaning it includes all variables.

If the function is 1 for specific combinations, we sum the minterms:

$$F(A,B) = \sum m(1,2,3)$$

Meaning: output is 1 for minterms m_1, m_2 , and m_3 .

Example: Given the truth table output is 1 at decimal positions 1 and 3:

$$F(A,B) = \sum m(1,3) = \bar{A}B + AB$$

6.3 Product of Sums (POS)

The Product of Sums form expresses a Boolean function as a product (AND) of sum (OR) terms.

General Form

$$F = (A + B)(\bar{A} + C)(B + \bar{C})$$

In POS, each sum term represents a condition when the output is 0.

Canonical POS (Product of Maxterms)

In canonical POS, every sum term is a maxterm, meaning it includes all variables.

If the function is 0 for specific combinations, we use maxterms:

$$F(A,B) = \Pi M(0,2)$$

Meaning: output is 0 at maxterms M_0 and M_2 .

Example: Given the function output is 0 at decimal positions 0 and 2:

$$F(A,B) = \Pi M(0,2) = (A + B)(\bar{A} + B)$$

6.4 Conversion Between SOP and POS

A Boolean function can be expressed in either SOP or POS form. Conversion is based on truth table positions.

Conversion Steps (SOP → POS)

1. Write the canonical SOP: $F = \Sigma m(1,3)$
2. Identify missing minterms: Total minterms = $\{0,1,2,3\}$ Missing = $\{0,2\}$
3. Write the zero positions as maxterms: $F = \Pi M(0,2)$

This is based on the rule: $\Sigma m(i) = \Pi M(\overline{\{i\}})$

Conversion Steps (POS → SOP)

1. Write the canonical POS: $F = \Pi M(0,2)$
2. Identify missing maxterms: Missing = $\{1,3\}$
3. Write as SOP: $F = \Sigma m(1,3)$

6.5 Normal vs Standard Forms

Boolean expressions can be written in Normal Form and Standard Form.

Normal Form

In normal SOP or normal POS, the terms may NOT contain all variables.

Example:

$$F = AB + C$$

(Variable C is alone, others are missing)

This is not canonical because some terms don't use all variables.

Standard Form

In standard SOP or POS, each term must contain all variables in either complemented or un-complemented form.

Example:

$$F = \bar{A}B + A\bar{B} + AB$$

Every term includes A and B $\rightarrow$ Standard form.

Difference

- Normal form: simplified, not all variables present in each term.
- Standard form: canonical, all variables present in each term.

Conclusion

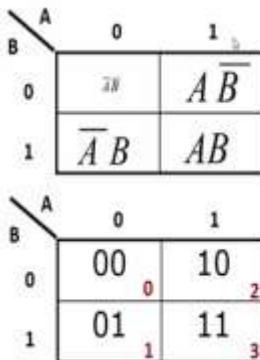
Canonical and standard forms provide a systematic way to express Boolean functions using minterms and maxterms. SOP represents the function when output is 1, while POS represents the function when output is 0. These forms enable the direct implementation of digital functions using AND-OR or OR-AND structures. Understanding these concepts is essential for digital logic design and circuit simplification.

CHAPTER 7 – KARNAUGH MAP (K-MAP) SIMPLIFICATION

Karnaugh maps, or K-maps, are often used to simplify logic problems with 2, 3 or 4 variables.

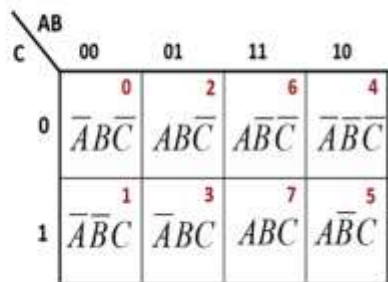
Cell = 2^n , where n is a number of variables

For the case of 2 variables, we form a map consisting of $2^2=4$ cells



3 variables K-map

Cell = $2^3=8$

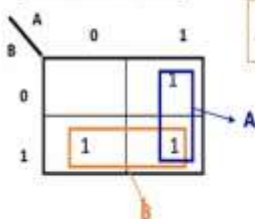


7.1 Examples and Minimization

OR Gate:



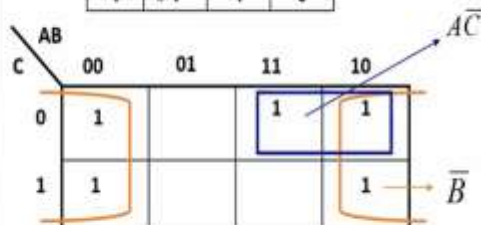
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

$F(A,B,C) = \sum (0, 1, 4, 5, 6)$

$\bar{B} + A\bar{C}$



CHAPTER 8 – COMBINATIONAL LOGIC CIRCUITS

8.1 Introduction

A Combinational Logic Circuit is a digital circuit in which the output depends only on the present inputs and not on previous inputs or memory elements.

These circuits perform logical operations using logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR.

Key characteristics:

- No feedback or storage element.
- Output changes immediately with input change.
- Based on Boolean algebra.
- Used to perform arithmetic, comparison, encoding, multiplexing, etc.

Mathematically: $\text{Output} = f(\text{Input1}, \text{Input2}, \dots \text{Inputn})$

Examples: Adder circuits, Multiplexers, Decoders, Comparators.

8.2 Properties of Combinational Circuits

Combinational circuits have the following fundamental properties:

1. Memory-less System
 - No flip-flops or latches.
 - Output depends only on present input values.
2. Deterministic Operation
 - For a given set of inputs, the output is always the same.
 - No uncertainty or delay due to feedback.
3. Direct Input-Output Relationship
 - They can be expressed as Boolean functions.
4. Parallel Operation
 - All outputs are generated simultaneously based on inputs.

5. Propagation Delay

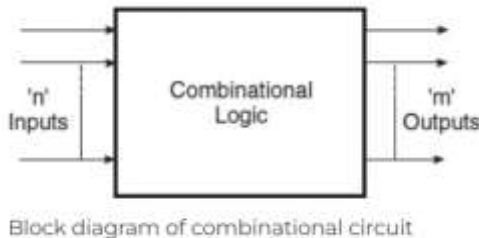
- Output changes after a small delay caused by gates.
- Total delay = sum of individual gate delays.

6. No Clock Requirement

- Works without clock signals, unlike sequential circuits.

8.3 Block Diagram Representation

A combinational circuit can be represented using a simple block diagram:



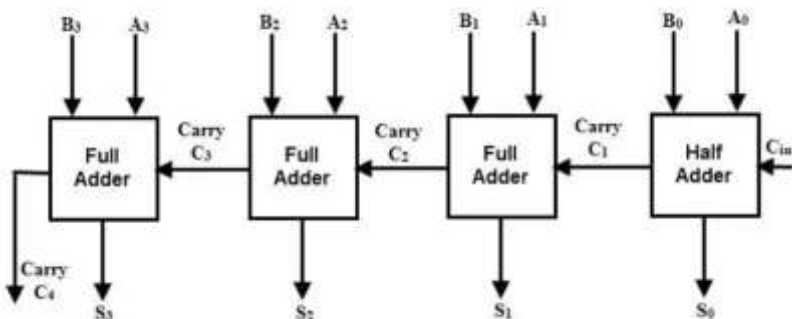
General representation:

- An input block containing n input variables.
- A logic block made from logic gates performing Boolean operations.
- An output block producing m output variables.

Example:

- 4-bit Adder: inputs (A_3-A_0 , B_3-B_0), output (S_3-S_0 , Carry).

Block diagrams help in understanding system behavior without showing internal gate structure.



8.4 Design Steps

Designing a combinational circuit involves systematic steps:

Step 1: Problem Definition

Understand the required relationship between inputs and outputs.

Step 2: Identify Inputs and Outputs

Determine:

- How many input variables?
- How many output variables?

Step 3: Input-Output Truth Table

Create a truth table showing all possible input combinations and their desired outputs.

Step 4: Boolean Expression

Write Boolean expressions for output variables from truth table using:

- Sum of Products (SOP)
- Product of Sums (POS)

Step 5: Simplify the Expression

- Boolean algebra
 - Karnaugh Map (K-Map)
 - Quine-McCluskey method
- to reduce the logic expression.

Step 6: Logic Gate Implementation

Draw the circuit diagram using logic gates:

- AND
- OR
- NOT
- NAND, NOR (if using universal gates)

Step 7: Verification

Check correctness by:

- Simulation
- Re-evaluating truth table
- Timing analysis

8.5 Applications

Combinational circuits are widely used in digital systems:

1. Arithmetic Circuits

- Adder
- Subtractors
- ALUs

2. Data Routing & Selection

- Multiplexers (MUX) & Demultiplexers (DEMUX)
- Encoders & Decoders

3. Code Conversion

- Binary to Gray
- BCD to 7-segment decoder
- ASCII to Binary

4. Comparison & Control

- Magnitude Comparators
- Priority Encoders
- Logic Control Units

5. Memory Addressing

- Address Decoding in microprocessors
- I/O selection logic & Chip enable logic

6. Display Systems

- Seven-segment display controllers
- LED matrix drivers

7. Communication Systems

- Error detection circuits (Parity generators)
- CRC logic & Digital modulation logic

CHAPTER 9 – ADDERS

9.1 Introduction

An **Adder** is a **Combinational Logic Circuit** that performs **binary addition**. It adds two or more binary numbers and gives **Sum** and **Carry** as outputs. Adders take two or more binary inputs and produce a binary sum and, in some cases, a carry output. They are widely used in arithmetic logic units (ALUs), processors, and digital systems for performing addition operations

Binary addition rules:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10 \text{ (Sum = 0, Carry = 1)}$$

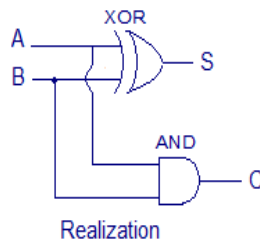
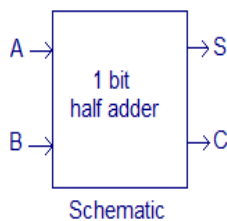
Based on functionality and complexity, adders can be categorized into different types.

9.2 Half Adder

A **Half Adder (HA)** adds two single-bit binary numbers and generates **Sum** and **Carry**.

Inputs		Outputs	
A	B	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Truth table



K-map simplification for carry and sum

For carry

A \ B	0	1
0	0	0
1	0	1

$$\text{Carry} = AB$$

For sum

A \ B	0	1
0	0	1
1	1	0

$$\begin{aligned} \text{Sum} &= \overline{A}B + A\overline{B} \\ &= A \oplus B \end{aligned}$$

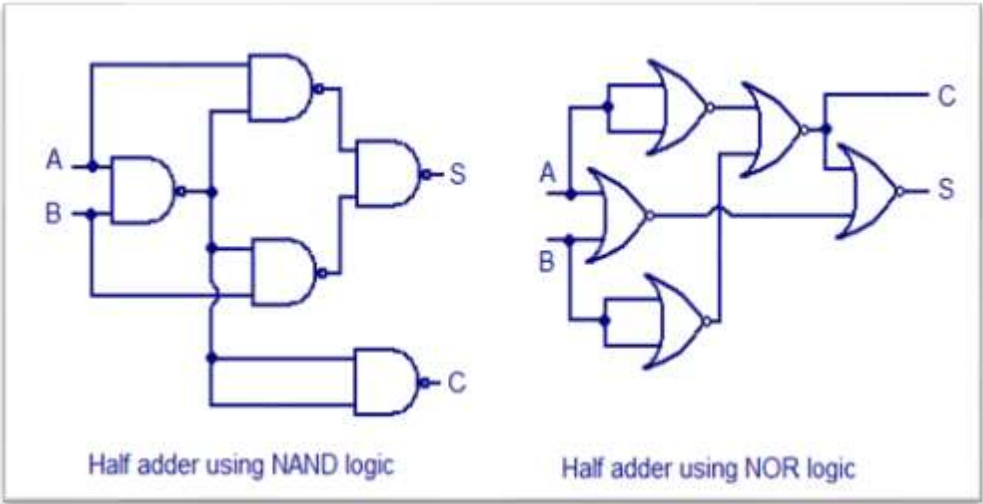
Inputs: A, B

Outputs: S, C

Boolean Expressions:

- Sum: $S = A \oplus B$
- Carry: $C = A \cdot B$

Type	Gates Used	Notes	Common Use
Basic Half Adder	XOR + AND	Standard type	Teaching, Digital circuits
NAND-only HA	NAND	Universal gate design	Hardware simplification
NOR-only HA	NOR	Alternative universal design	CMOS design
XOR-AND HA	XOR + AND	Simplest hardware	IC implementation



Limitation:

- It cannot add a carry from a previous stage, so it is used for simple 1-bit addition only.

9.3 Full Adder

Adds **three 1-bit numbers** (A, B, and Carry-in) and Produces

Inputs			Outputs	
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

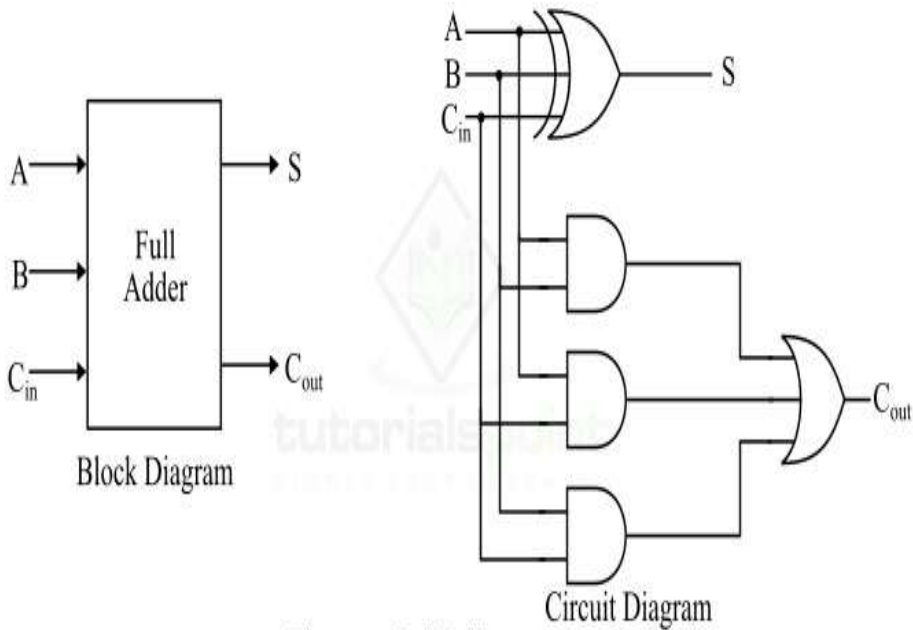
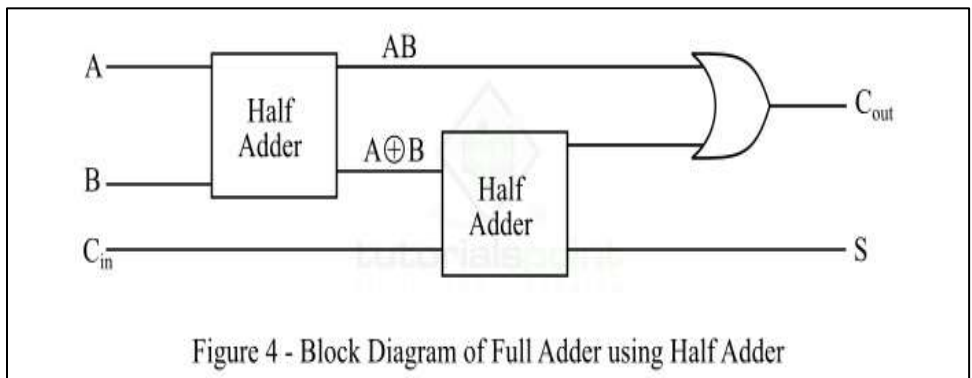
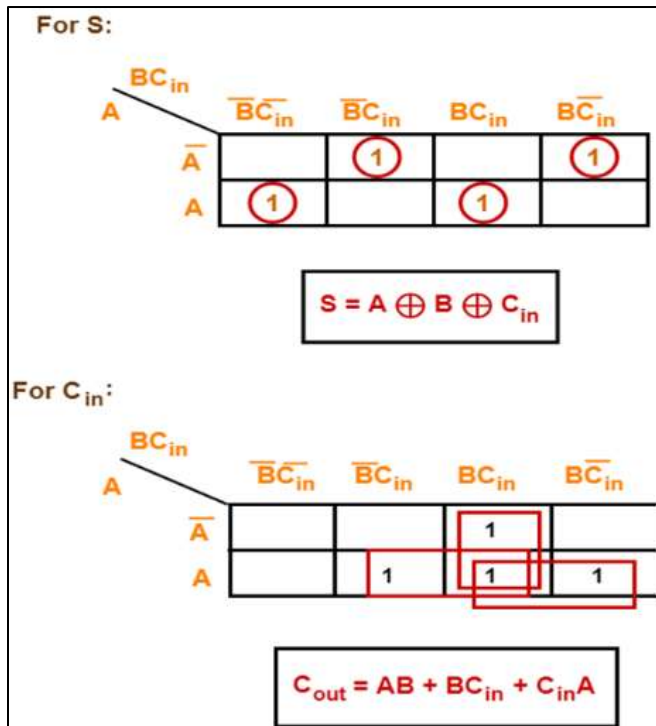


Figure1 - Full Adder



Advantages:

- Can add multi-bit binary numbers when multiple Full Adders are cascaded.

Multi-bit Adders

Adder Type	Number of Bits	Description
4-bit Adder	4 bits	4 Full Adders connected in series
8-bit Adder	8 bits	8 Full Adders connected
16-bit Adder	16 bits	16 Full Adders connected
32-bit Adder	32 bits	Used in processors and ALUs

Types of Multi-bit Adders

Ripple Carry Adder (RCA) – simple, but slow (carry ripples through).

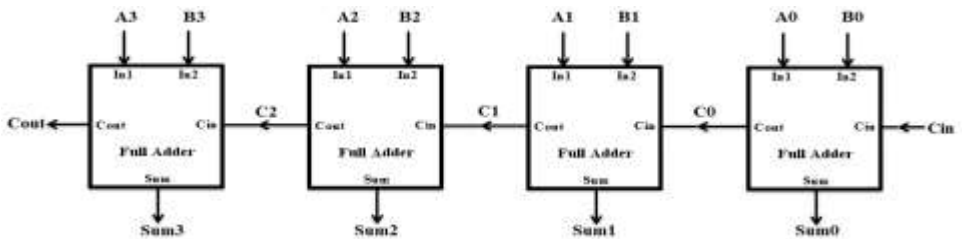
Carry Look-Ahead Adder (CLA) – faster, uses carry prediction.

Carry Select Adder (CSA) – faster than RCA, uses precomputed results.

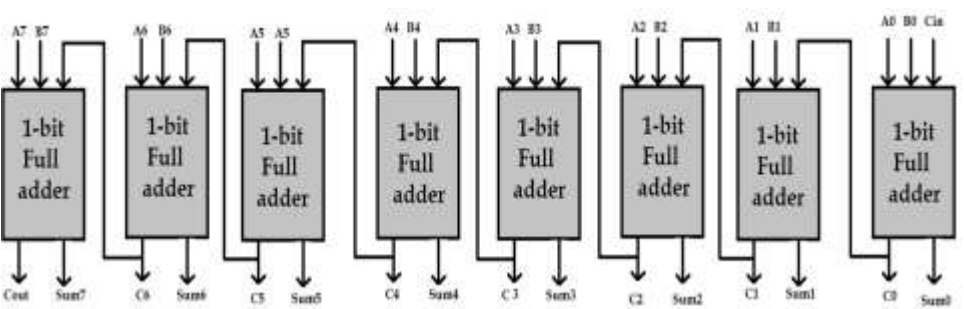
Carry Skip Adder (CSkA) – skips carry through blocks for speed.

Parallel Prefix Adder (e.g., Kogge-Stone) – used in modern CPUs.

4-bit Adder



8-bit Adder

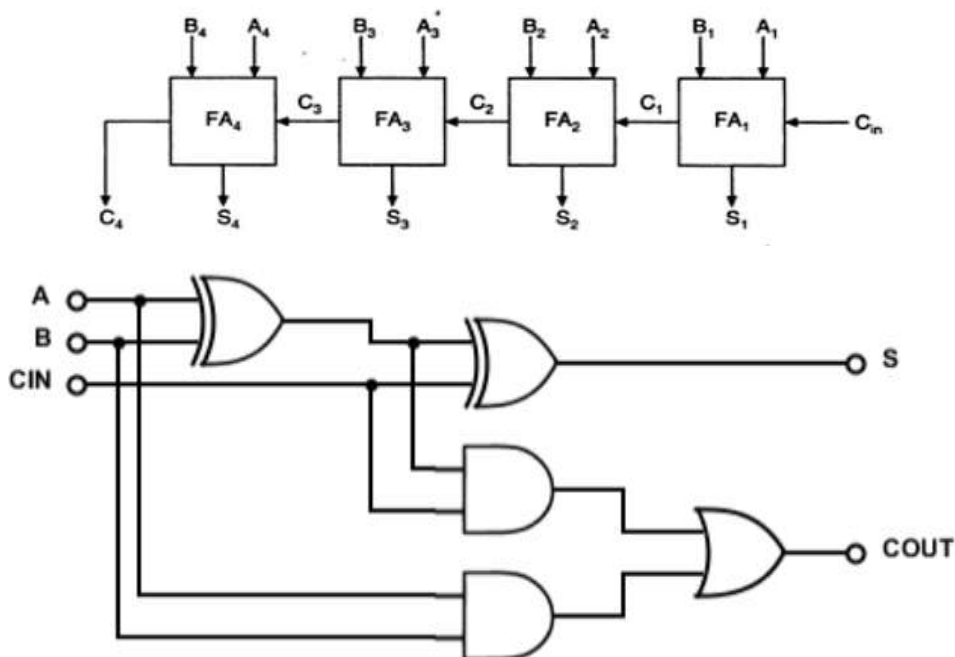


9.4 Ripple Carry Adder

A **Ripple Carry Adder** is made by **connecting multiple Full Adders in series** to add multi-bit binary numbers. The **carry-out** from each stage becomes the **carry-in** to the next

Example:

For adding two 4-bit numbers ($A_3A_2A_1A_0$ and $B_3B_2B_1B_0$), you connect **4 Full Adders** in sequence.



Propagation Delay: If one Full Adder has delay t , total delay = $n \times t$.

Problem:

The carry has to **ripple (propagate)** through each stage. So, **delay increases** with the number of bits → slower speed.

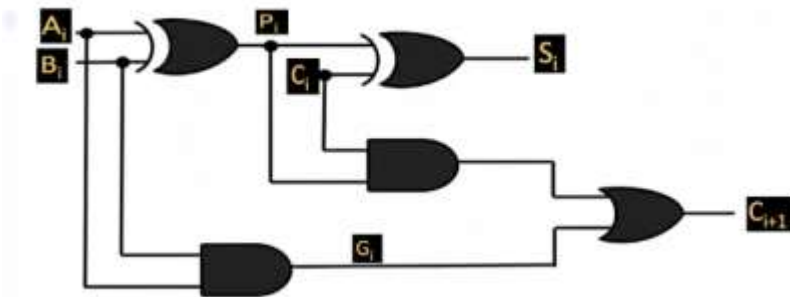
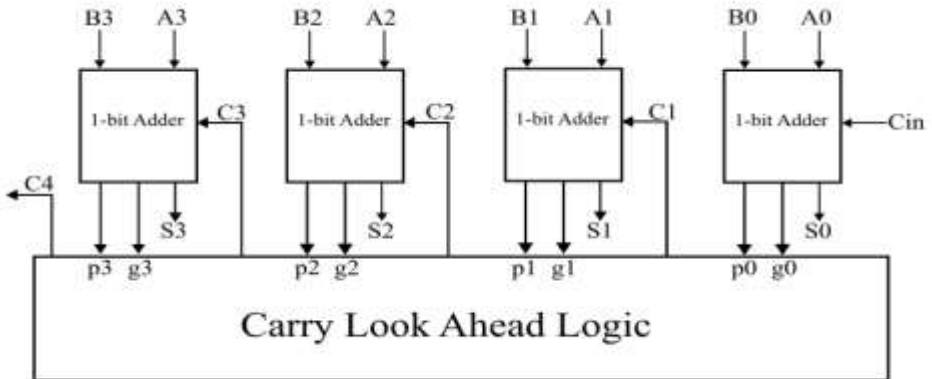
PPA summary:

- **Good for:** Low-power, low-cost designs (simple microcontrollers).
- **Not good for:** High-speed systems.

Example use: Simple calculators or small embedded systems.

9.5 Carry Look-Ahead Adder

A **Carry Look-Ahead Adder (CLA)** is a high-speed digital adder that **computes carry signals in advance** using *generate* and *propagate* functions, instead of waiting for them to ripple through each bit. This reduces the overall addition time and makes the adder much faster than a ripple-carry adder.



$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$G_n = A_n B_n$$

$$P_n = A_n \oplus B_n$$

Carry propagate $\rightarrow 2n$ gate level (for 4-bit adder $\rightarrow 2 \times 4 = 8$)

$P_i \rightarrow$ Carry propagate = $G_i \rightarrow$ Carry GENERATE

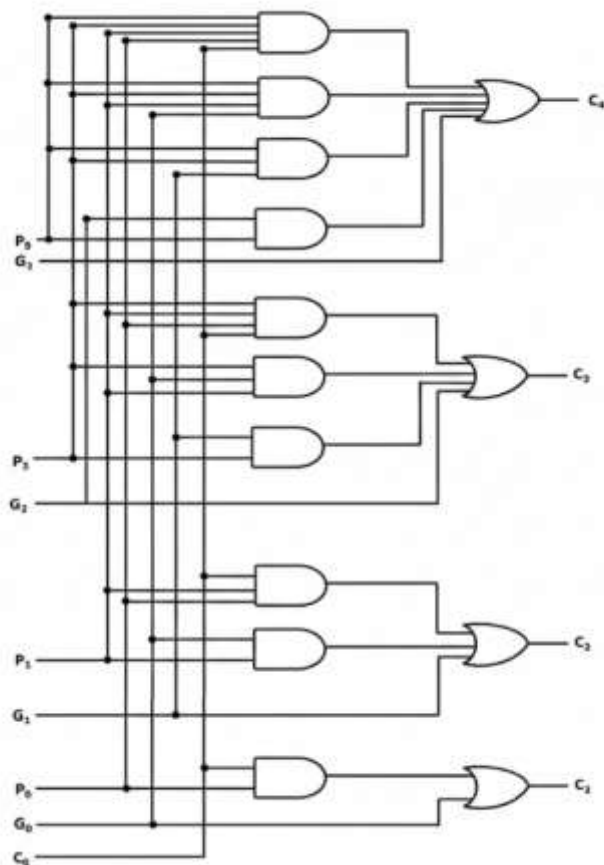
$$P_i = A_i \oplus B_i$$

$$S_i = P_i \oplus C_i$$

$$G_i = A_i B_i$$

$$C_{i+1} = G_i + P_i C_i$$

$$A_i \quad S_i \quad C_{i+1}$$



Advantages of Carry Look-Ahead Adder

- **Very fast addition**

Carries are calculated in parallel, reducing delay.

- **Removes ripple-carry delay**

No need for the carry to move from bit to bit.

- **Better performance for wide adders**

Useful in 8-bit, 16-bit, 32-bit ALUs.

- **Improves CPU speed**

Widely used in processors and high-speed arithmetic circuits

Disadvantages of Carry Look-Ahead Adder

- **More hardware complexity**

Requires extra gates for generate/propagate equations.

- **Increased wiring**

Carry logic becomes complicated for large bit-widths.

- **Higher power consumption**

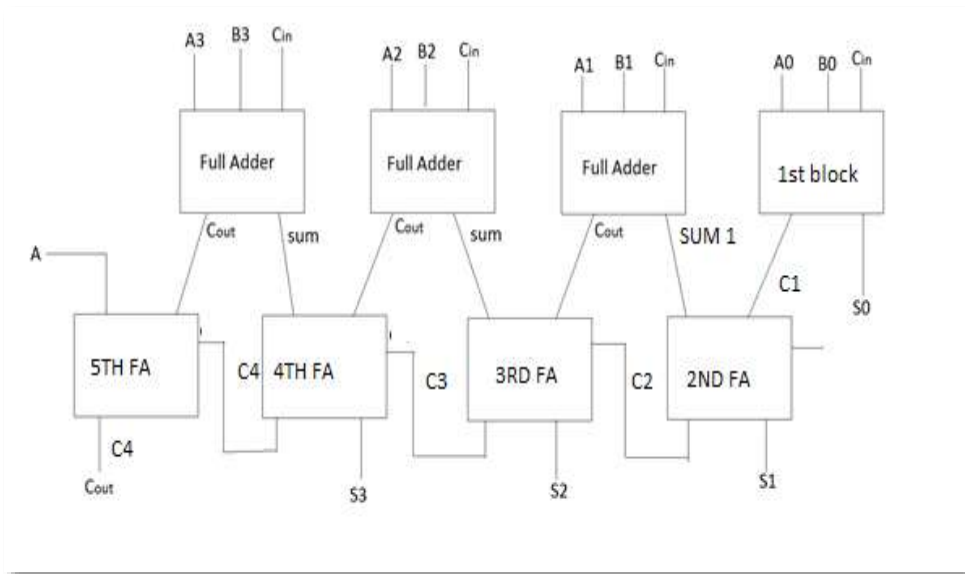
Due to more logic gates operating in parallel.

- **Difficult to implement for very large adders**

Often needs hierarchical CLAs (like 16-bit or 32-bit CLA with blocks).

9.6 Carry Save Adder

A Carry Save Adder is a high-speed adder used to add three or more numbers at the same time. Instead of propagating carry from one bit to the next, the CSA saves the carry in a separate register for the next stage. This avoids carry propagation delay and makes addition extremely fast.



Advantages

- Very high speed because it avoids carry propagation
- Can add multiple numbers (3 or more) simultaneously
- Ideal for partial product reduction in multipliers
- Simple hardware: only uses 3-input full adders
- Reduces critical path delay in arithmetic circuits

Disadvantages

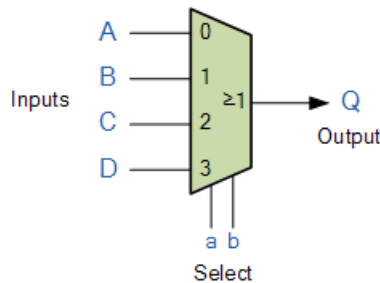
- Cannot produce the final sum directly
- Outputs Sum + Carry, requiring a final adder (CLA or CPA) at the end
- Not suitable for adding only two numbers
- Uses more hardware in multi-stage structures

CHAPTER 10 – MULTIPLEXERS (MUX)

10.1 Introduction

A **Multiplexer (MUX)** is a digital circuit that **selects one output from multiple input signals** based on the combination of **select lines**. It allows **many inputs to share a single output line**, by choosing which input is passed to the output at a given time.

- **Many inputs → One output**
- **Controlled by select lines**



In digital systems, multiplexers are used to:

- Reduce the number of communication lines
- Select information from multiple sources
- Implement logic functions

A multiplexer with n select lines can select from 2^n input lines.

General formula:

If number of input lines = 2^n , then number of select lines = n .

ADVANTAGES of MUX

- Allows multiple signals to share a **single communication line**
- **Reduces wiring**
- Helps in building **complex digital circuits**
- Can be used to implement **Boolean functions**
- High efficiency for data routing

DISADVANTAGES of MUX

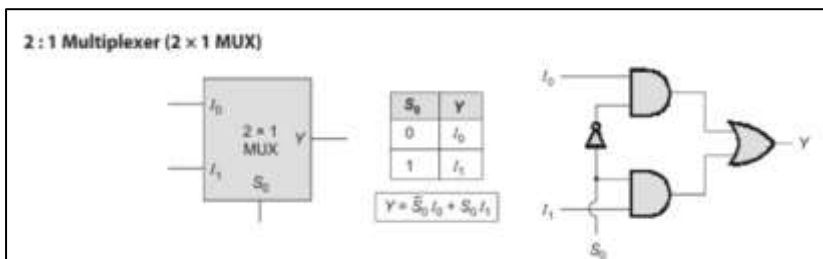
- Limited by the number of **select lines**
- Sensitive to noise and glitches
- Large multiplexers need cascading → more delay
- Increased hardware for higher bit-widths

10.2 Types of Multiplexer

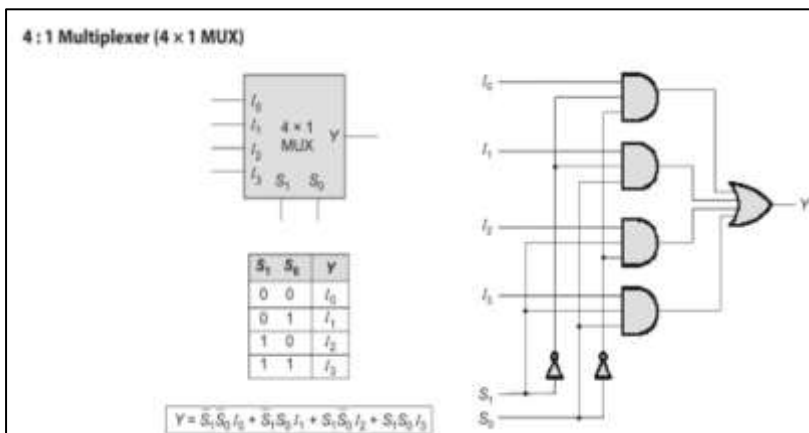
- 2-to-1 MUX
- 4-to-1 MUX
- 8-to-1 MUX
- 16-to-1 MUX
- 32-to-1 MUX
- Tree MUX (cascaded structure)
- Analog MUX
- Digital MUX

(The multiplexers described below are only a few basic types. Larger multiplexers can be constructed by connecting these smaller MUX circuits in a hierarchical structure.)

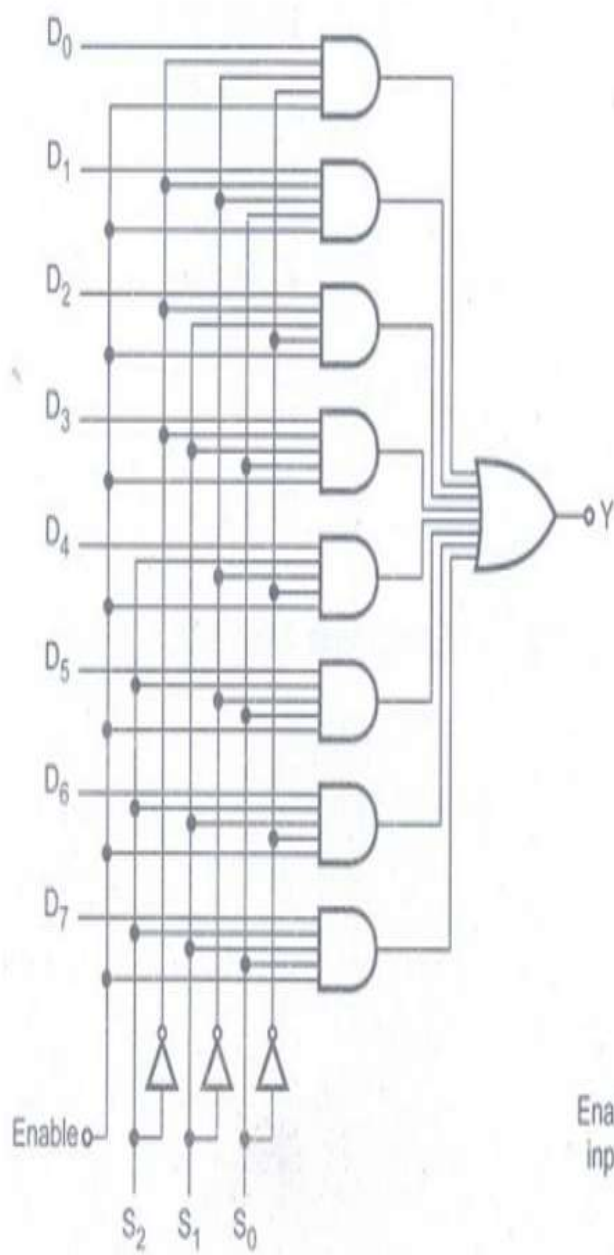
2-to-1 MUX



4-to-1 MUX



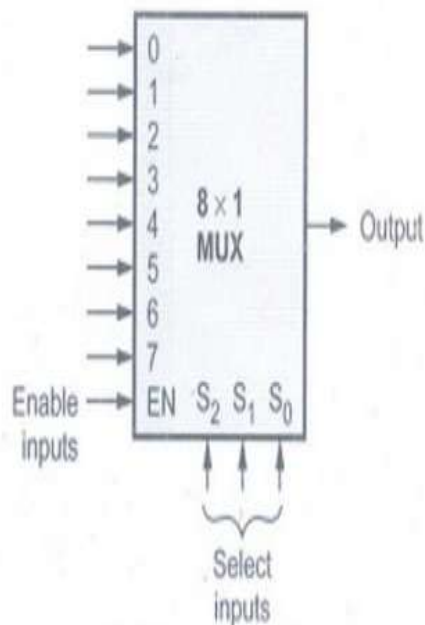
8-to-1 MUX



(a) Logic diagram

S_2	S_1	S_0	Y
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

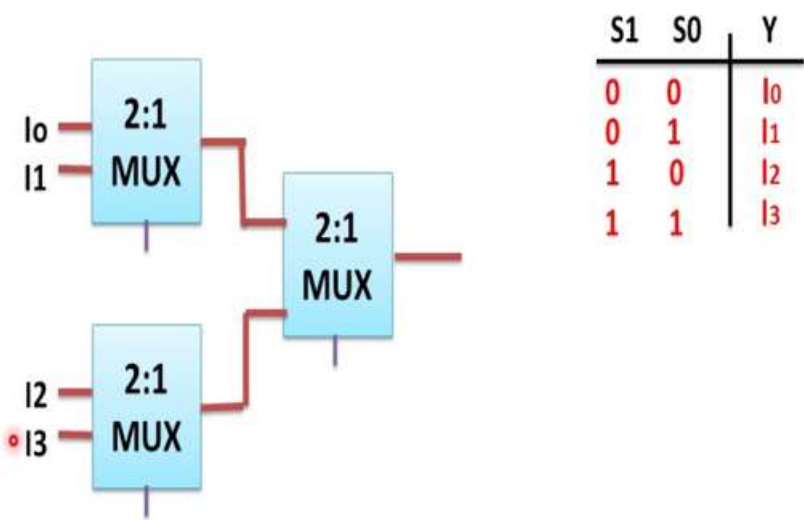
(b) Function table



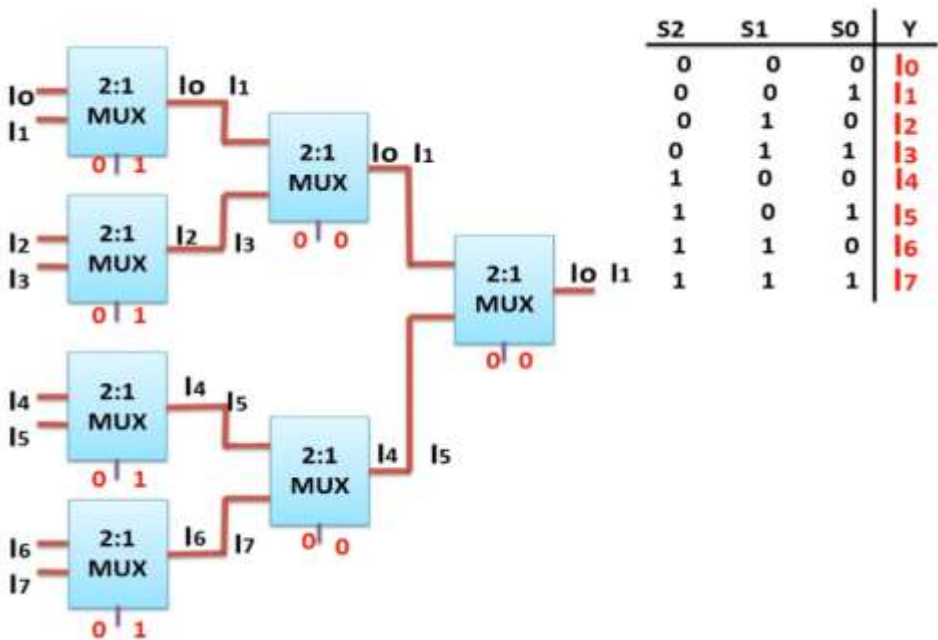
(c) Logic symbol

10.3 Implementation Using MUX

IMPLEMENTING 8:1 MUX USING 2:1 MUX



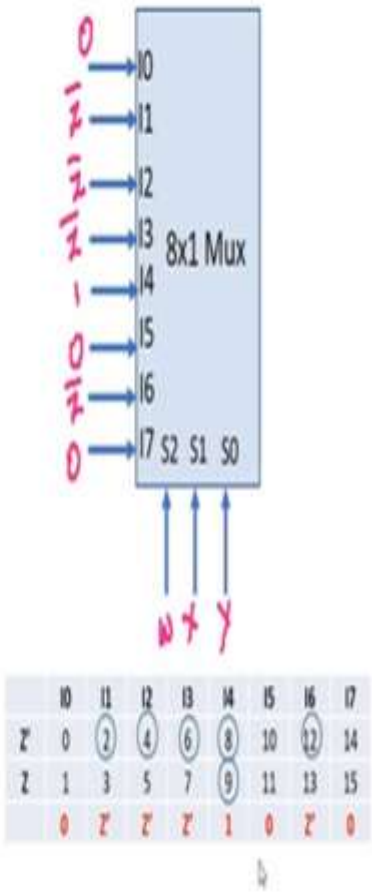
IMPLEMENTING 4:1 MUX USING 2:1 MUX



LOGIC IMPLEMENTATION USING MUX

$F = WX'Y' + XY'Z' + W'YZ'$

W	X	Y	Z	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0



EXERCISE:

- IMPLEMENT THE LOGIC GATES USING MUX
- IMPLEMENT THE FULL ADDER USING MUX

10.4 Applications of Multiplexers

Multiplexers have a wide range of applications in digital systems:

1. Data Routing

- Select one data input from multiple sources
- Used in CPUs, communication systems

2. Function Generator

- Implement Boolean functions
- Generate logic operations when used with select lines

3. Signal Multiplexing

- Combine several analog/digital signals onto one line
- Used in data transmission

4. Arithmetic Circuits

- Full Adder implementation
- ALU design in processors

5. Control Systems

- Control signal selection in microprocessors
- Program counter selection logic

6. Memory Management

- Address selection
- Read/Write line selection in RAM/ROM

7. Code Converters

- Binary to Gray, binary to BCD conversion

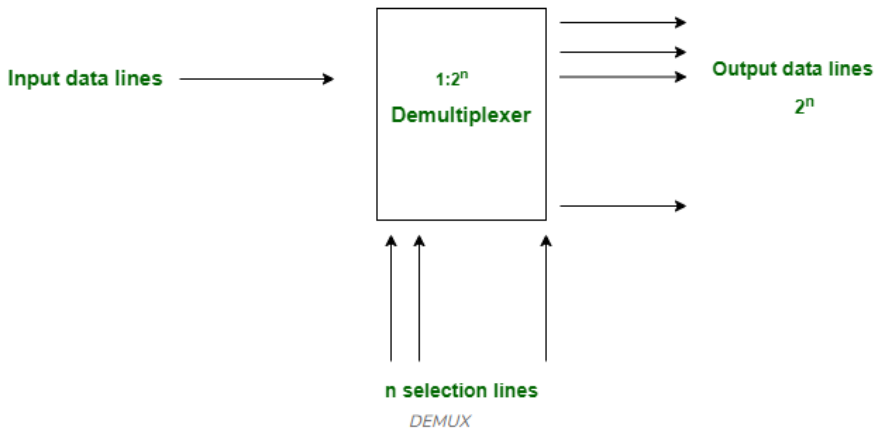
8. Parallel-to-Serial Converter

- Converts parallel data to serial form using multiplexed selection

CHAPTER 11 – DEMULTIPLEXERS (DEMUX)

11.1 Introduction

A Demultiplexer (DEMUX) is a combinational logic circuit that performs the reverse operation of a multiplexer. It takes a single input and distributes it to one of many output lines based on the values of selection lines. It is also called a data distributor because it transfers a single input data to one of the multiple output lines.



The selection lines decide which output will receive the input value, while all other outputs remain inactive (usually logic '0').

If number of selection lines = n , then number of output lines = 2^n .

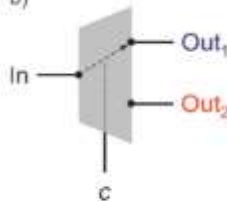
11.2 1:2, 1:4, 1:8 DEMUX

1:2 DEMUX

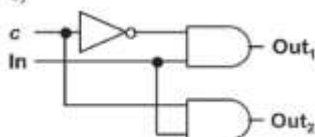
a)

In	c	Out ₁	Out ₂
0	0	0	0
1	0	1	0
0	1	0	0
1	1	0	1

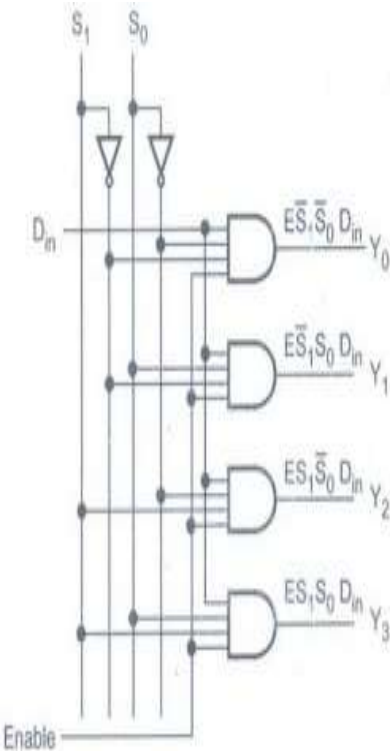
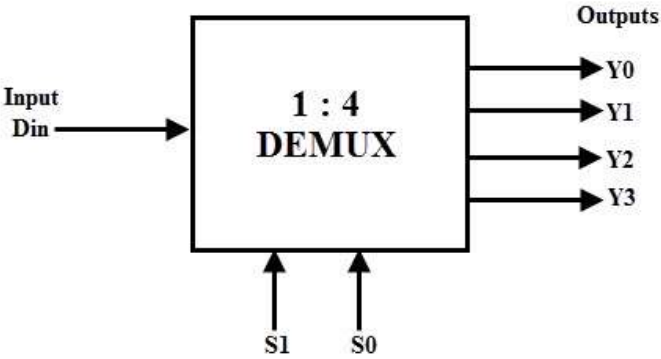
b)



c)



1:4 DEMUX



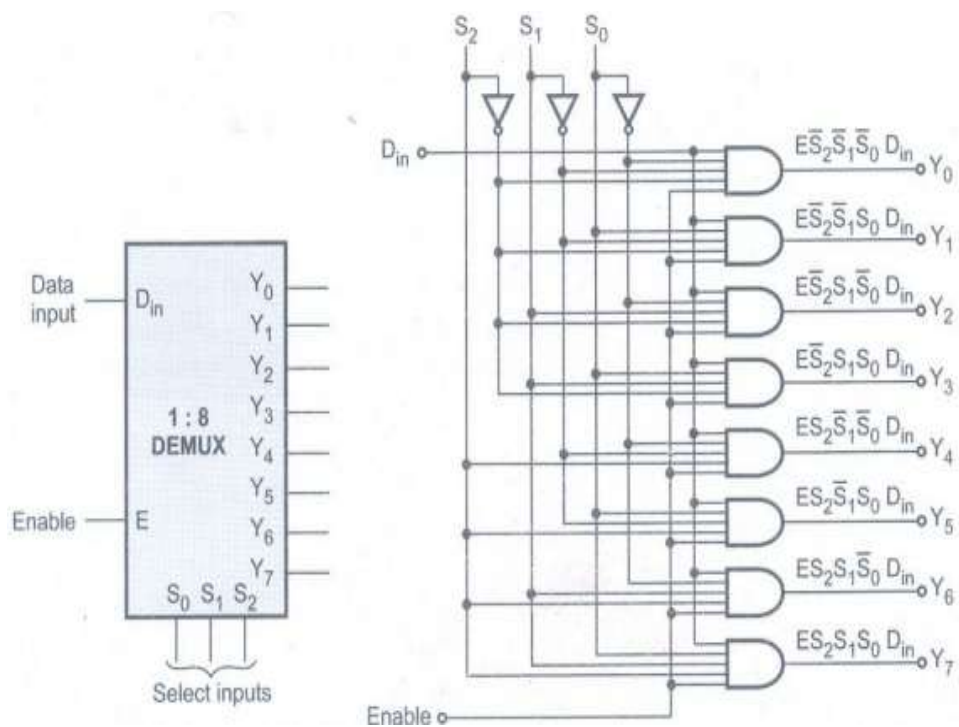
(a) Logic diagram

Enable (E)	S ₁	S ₀	D _{in}	Y ₀	Y ₁	Y ₂	Y ₃	
0	X	X	X	0	0	0	0	
1	0	0	0	0	0	0	0	D _{in} is routed to Y ₀ Y ₀ = E $\bar{S}_1 \bar{S}_0$ D _{in}
1	0	0	1	1	0	0	0	
1	0	1	0	0	0	0	0	D _{in} is routed to Y ₁ Y ₁ = E $\bar{S}_1 S_0$ D _{in}
1	0	1	1	0	1	0	0	
1	1	0	0	0	0	0	0	D _{in} is routed to Y ₂ Y ₂ = E S ₁ $\bar{S}_0$ D _{in}
1	1	0	1	0	0	1	0	
1	1	1	0	0	0	0	0	D _{in} is routed to Y ₃ Y ₃ = E S ₁ S ₀ D _{in}
1	1	1	1	0	0	0	1	

(b) Function table for 1 : 4 demultiplexer

Fig. 1.18.2

1:8 DEMUX



Enable		Select inputs			Outputs							$E \bar{S}_2 \bar{S}_1 \bar{S}_0 D_{in}$ $E \bar{S}_2 \bar{S}_1 S_0 D_{in}$ $E \bar{S}_2 S_1 \bar{S}_0 D_{in}$ $E \bar{S}_2 S_1 S_0 D_{in}$ $E S_2 \bar{S}_1 \bar{S}_0 D_{in}$ $E S_2 \bar{S}_1 S_0 D_{in}$ $E S_2 S_1 \bar{S}_0 D_{in}$ $E S_2 S_1 S_0 D_{in}$
E	S_2	S_1	S_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7	
0	X	X	X	0	0	0	0	0	0	0	0	
1	0	0	0	D_{in}	0	0	0	0	0	0	0	
1	0	0	1	0	D_{in}	0	0	0	0	0	0	
1	0	1	0	0	0	D_{in}	0	0	0	0	0	
1	0	1	1	0	0	0	D_{in}	0	0	0	0	
1	1	0	0	0	0	0	0	D_{in}	0	0	0	
1	1	0	1	0	0	0	0	0	D_{in}	0	0	
1	1	1	0	0	0	0	0	0	0	D_{in}	0	
1	1	1	1	0	0	0	0	0	0	0	D_{in}	

11.3 Cascading DEMUX

When a large number of outputs are required, multiple DEMUXes can be connected together in a cascading (tree) structure.

For example:

- Two 1:4 DEMUXes can create a 1:8 DEMUX
- Four 1:4 DEMUXes can create a 1:16 DEMUX

Advantages of cascading:

- Modular design
- Easy expansion
- Reduces cost for large systems

In cascading, one DEMUX drives the enable input of other DEMUXes, distributing the selection process in multiple stages.

11.4 Applications

Demultiplexers are widely used in digital communication and computer systems.

1. Data Distribution

- Distributes one data source to multiple destinations
- Used in network routers and data link layers

2. Serial to Parallel Conversion

- Converts serial data into parallel data using DEMUX selection

3. Memory Addressing

- Used for RAM/ROM address decoding
- Selection of memory banks

4. Digital Display Systems

- Driving seven-segment displays
- LED display control

5. Communication Systems

- De-multiplexing of time-division multiplexed (TDM) signals
- Separating encoded signals into individual channels

6. Arithmetic Circuits

- Routing data to different parts of ALU
- Selective loading of registers

7. Control Logic

- Selective operation of devices based on control signals
- Used in FPGA and microprocessor architectures

CHAPTER 12 – ENCODERS AND DECODERS

12.1 Encoders

An encoder is a combinational circuit that converts 2^n input lines into n output lines. It performs the opposite operation of a decoder — it encodes the input data into a smaller number of output bits.

Purpose: An Encoder is used to convert multiple input signals into a smaller binary code — it compresses information

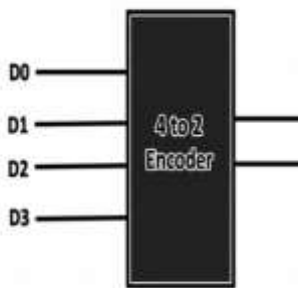
Key Points:

1. Converts data from 2^n inputs n outputs.
2. Each input line represents a unique binary code.
3. Used to reduce the number of data lines.
4. Example: 8-to-3 encoder (8 inputs, 3 outputs).
5. If multiple inputs are active, priority encoder is used to assign priority.
6. Commonly used in keyboard encoding, data compression, and memory addressing.

Features

- Converts **multiple inputs** into a **smaller binary output**
- Only **one input** should be HIGH at a time
- Output is a **binary representation** of active input
- Reduces number of wires and data lines

4 to 2 Encoder



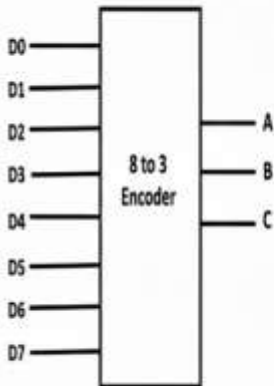
4 line to 2 line Encoder

Truth Table

D0	D1	D2	D3	A	B
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

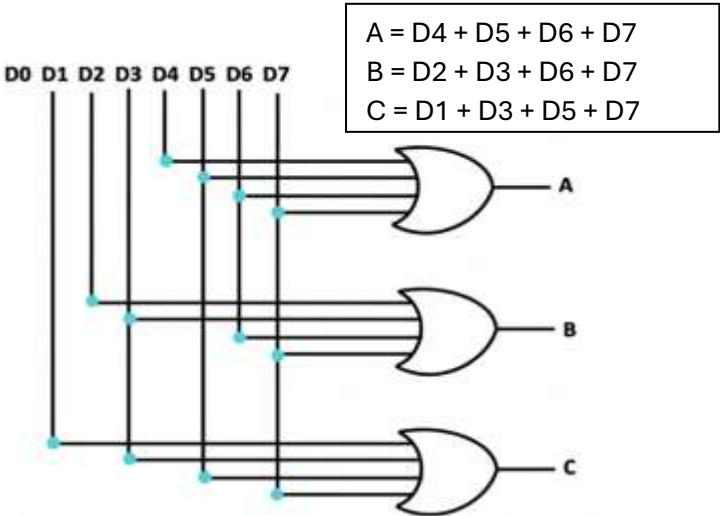
$A = D2 + D3$
 $B = D1 + D3$

8 to 3 Encoder



Truth Table

D0	D1	D2	D3	D4	D5	D6	D7	A	B	C
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	1	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1



12.2 Types of Encoders

- 1. Priority Encoder
- 2. Binary Encoder
- 3. Decimal to BCD Encoder
- 4. Octal to Binary Encoder

Type	Inputs	Outputs	Special Feature	Example Use
Binary Encoder	2^n	n	Basic type	Data compression
Priority Encoder	2^n	n	Handles multiple active inputs	Interrupt systems
Decimal to BCD Encoder	10	4	Decimal $\rightarrow$ Binary	Calculator, keypad
Octal to Binary Encoder	8	3	Octal $\rightarrow$ Binary	Digital conversion systems

1. Priority Encoder

A priority encoder assigns priority to inputs. If two or more inputs are active at the same time, it encodes only the highest-priority input.

Example:

4-to-2 Priority Encoder: If D_2 and D_3 are both 1, the output represents D_3 because it has higher priority.

2. Binary Encoder

A binary encoder converts 2^n input lines into an n-bit binary code. It is the simplest and most common type of encoder.

Example:

8-to-3 Binary Encoder:

- Inputs: $D_0 - D_7$ (8 inputs)
- Outputs: Y_2, Y_1, Y_0 (3 bits)

3. Decimal to BCD Encoder

A Decimal-to-BCD encoder converts decimal input (0–9) into a 4-bit Binary-Coded Decimal (BCD) output.

Example:

Decimal 2 → BCD 0010

Decimal 9 → BCD 1001

4. Octal to Binary Encoder

An Octal-to-Binary encoder converts 8 input lines (octal digits) into a 3-bit binary output.

Octal Input	Binary Output
D ₀	000
D ₁	001
D ₂	010
D ₃	011
D ₄	100
D ₅	101
D ₆	110
D ₇	111

Logic Equations (Octal to Binary Encoder)

$$\begin{aligned} A &= D_4 + D_5 + D_6 + D_7 \\ B &= D_2 + D_3 + D_6 + D_7 \\ C &= D_1 + D_3 + D_5 + D_7 \end{aligned}$$

12.3 Decoders

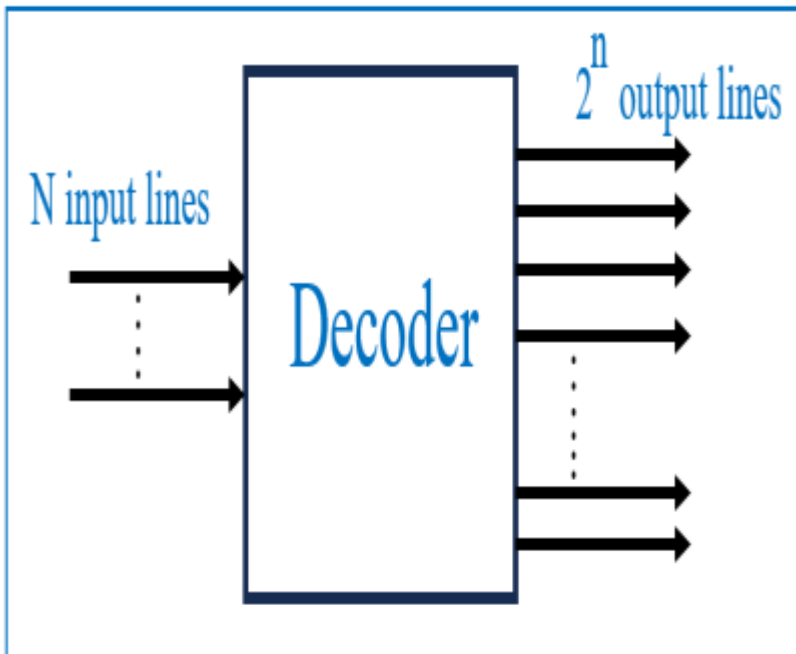
A decoder is a combinational circuit that converts n input lines into 2^n output lines. It performs the reverse operation of an encoder by decoding the binary input into a unique active output line.

Purpose

A decoder is used to convert binary input data into a specific output line, effectively expanding the information.

Key Points

- Converts data from n inputs $\rightarrow 2^n$ outputs.
- Only one output is active for each input combination.
- Used for data selection and memory address decoding.
- Example: 3-to-8 decoder (3 inputs, 8 outputs).
- Usually built using AND gates.
- Widely used in microprocessors, memory systems, and display devices.



12.5 Applications in Memory and Addressing

Encoders and decoders play a major role in digital memory systems and address selection units. They are used to convert binary addresses, enable memory blocks, and route data between different memory locations.

1. Address Decoding in Memory

Decoders are used to **translate binary address inputs** from the processor into **individual memory select signals**. For an **n-bit address**, a **2^n -line decoder** activates exactly **one memory location** at a time.

Example: A **4-to-16 decoder** activates one out of 16 memory blocks based on a 4-bit address.

2. Chip Select Logic

In complex memory systems (RAM/ROM), decoders generate **Chip Select (CS)** signals. Only the selected chip becomes active, while others remain disabled, preventing bus conflicts.

3. Memory Bank Selection

Encoders and decoders are used to **divide large memory arrays** into multiple banks. The decoder selects the **required bank**, while the encoder identifies which bank generated a request or interrupt.

4. I/O Port Addressing

During communication with peripheral devices, decoders generate **I/O device addresses**. This allows the CPU to send and receive data from a specific port.

5. Row and Column Selection (in DRAM)

In DRAM architecture:

- **Row Address Decoder** activates the correct row.
- **Column Address Decoder** selects the required column.

This enables fast access to memory cells using fewer address lines.

6. Interrupt Priority Encoding

Encoders are used to **assign priority levels** to interrupt requests from multiple devices. The CPU receives a **binary code** indicating the **highest-priority interrupt**, enabling efficient control.

7. Address Multiplexing

Encoders help in **address multiplexing**, where multiple input addresses are encoded to reduce the number of lines required for transmission.

CHAPTER 13 – COMPARATORS

Comparator

A **Comparator** is a **combinational circuit** that compares **two binary numbers** (A and B) and determines their **magnitude relationship**, giving one of the three possible results:

- $A > B$
- $A = B$
- $A < B$

Main Function

It **compares two input values** and produces **output signals** indicating whether A is **greater than**, **equal to**, or **less than** B.

Types of Comparators

1. 1-bit Comparator

- Compares two single-bit numbers (A and B).
- Outputs: $A > B$, $A = B$, $A < B$

2. 2-bit Comparator

- Compares two 2-bit binary numbers.

3. 4-bit Comparator

- Compares two 4-bit binary numbers
- (Implemented using IC **7485**)

Why Comparators Are Used

- Decision-making circuits
- Sorting operations
- Equality checking
- Sensor value comparison
- CPU digital operations
- Control and automation systems

CHAPTER 14 – MULTIPLIERS

A **Multiplier** is a **digital circuit** used to **multiply two binary numbers**. It performs multiplication similar to manual arithmetic, but implemented using **logic gates in hardware**.

In short: Multiplier = A circuit that performs binary multiplication.

Why Multipliers Are Used?

- For **fast arithmetic operations**
- Used in **processors (ALU)**
- Needed in **DSP (Digital Signal Processing)**
- Used in **image processing**, filters, robotics, and control systems
- Essential for **VLSI and microcontroller design**

Types of Binary Multipliers

1. **Array Multiplier**
 - Uses a grid (array) of adders
 - Simple structure and easy to design
2. **Booth Multiplier**
 - Used for signed multiplication
 - Reduces the number of partial products
3. **Carry-Save Multiplier**
 - Speeds up multiplication using carry-save adders
4. **Wallace Tree Multiplier**
 - Uses parallel reduction of partial products
 - Very fast for large bit-width operations

Advantages

- **High speed** compared to software multiplication
- **Parallel processing** of bits
- **Low latency** in signal processing
- **Efficient hardware implementation**

CHAPTER 15 – SEQUENTIAL LOGIC CIRCUITS

15.1 Introduction

A Sequential Logic Circuit is a digital circuit in which the output depends on both the present inputs and the past history of inputs. Unlike combinational circuits, sequential circuits contain memory elements (such as latches and flip-flops) that store information.

Sequential Circuit = Combinational Logic + Memory

Characteristics:

- Output depends on current input and stored data.
- Uses clock signals to synchronize operations.
- Can store and process sequences of data.
- Used in counters, registers, timers, state machines, memory, etc.

Examples: Flip-flops, Registers, Counters, FSM (Finite State Machines)

15.2 Clock and Timing Parameters

Sequential circuits are controlled using a clock signal. A clock is a periodic waveform that triggers changes in memory elements at specific time intervals.

Clock Signal

- Typically a square wave.
- Repeats every Clock Period (T).
- Frequency (f) = $1 / T$.

Important Timing Parameters

1. Clock Period (T):

Time duration of one complete clock cycle.

2. Clock Frequency (f):

Number of cycles per second (Hz).

$$f = \frac{1}{T}$$

3. Setup Time (Ts):

Minimum time for which input data must be stable before the clock edge.

4. Hold Time (Th):

Minimum time for which input data must remain stable after the clock edge.

5. Propagation Delay:

Time taken by signal to travel between logic elements.

6. Clock Pulse Width:

Duration for which clock stays HIGH (or LOW).

These parameters are important for timing analysis and reliable operation of sequential circuits.

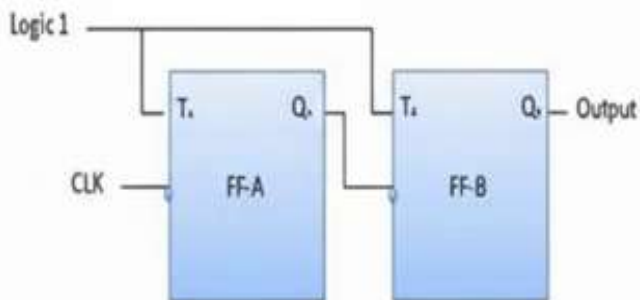
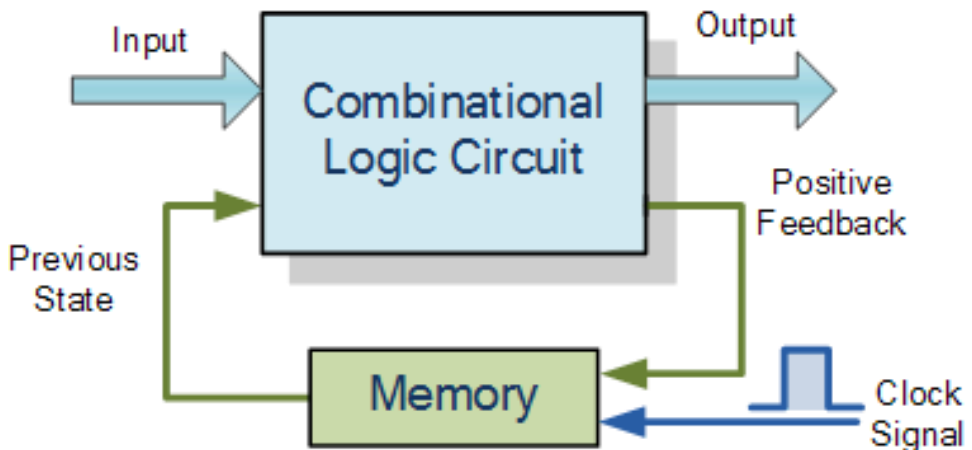
15.3 Difference Between Combinational and Sequential Circuits

Feature	Combinational Circuit	Sequential Circuit
Memory	No memory	Has memory
Output depends on	Present inputs only	Present + past inputs
Time dependence	No	Yes
Components	Logic gates	Logic gates + Flip-flops
Feedback	No	Yes (often)
Examples	Adder, Decoder, MUX	Counter, Register, FSM

15.4 Block Structure

A general **sequential logic circuit** contains three main parts:

1. **Combinational Logic Block**
 - Performs logical operations on input signals.
 - Generates next state values and outputs.
2. **Memory Elements**
 - Flip-flops, latches, registers.
 - Store the **current state** of the system.
3. **Clock Signal**
 - Synchronizes the transfer of data.
 - Controls when memory elements update.



15.3 LEVEL TRIGGERED vs EDGE TRIGGERED

LEVEL TRIGGERED

A circuit is level-triggered when it responds to the continuous level of a signal.

- If the enable/clock is HIGH (1) → the circuit keeps working.
If it is LOW (0) → the circuit stops.

Easy Example: A Latch is level-triggered. When Enable = 1, the latch keeps updating continuously.

Think Like This

- Level = as long as the door is open, you can walk.
- Circuit reacts the entire time the signal's level is high (or low).

EDGE TRIGGERED

A circuit is edge-triggered when it responds only at the moment of change in the signal.

It works only at:

- Rising edge (↑) → 0 → 1 transition
OR
- Falling edge (↓) → 1 → 0 transition

(Not continuous)

Easy Example: A Flip-Flop is edge-triggered. It updates only at the exact instant when the clock edge occurs.

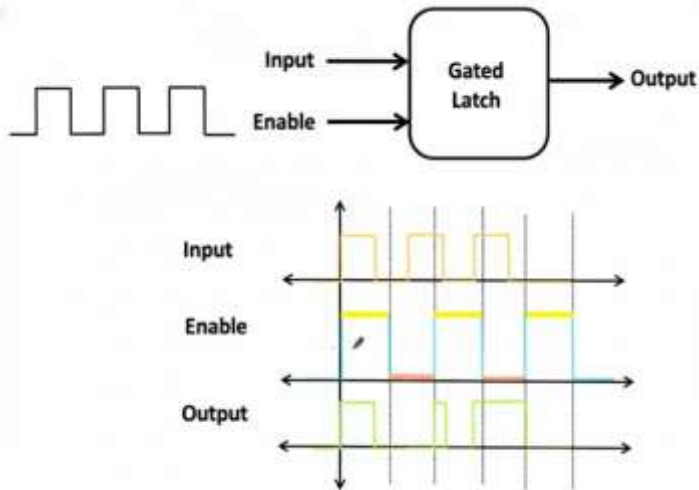
Think Like This

- Edge = only when the door opens or closes, you can enter.
- After the instant passes, nothing changes.

CHAPTER 16 – LATCHES

16.1 Latch

A latch is a basic memory device that stores one bit (0 or 1) of data and changes its output only when its enable/input level is active. It is level-triggered, meaning it responds to HIGH or LOW levels, not edges.



Do Latches Need a Clock?

- NO. Latches do not use a clock signal.
- Latches are level-triggered devices.

They work using an Enable (EN) signal, not a clock.

When Enable = 1, the latch becomes transparent (output follows input).

When Enable = 0, the latch holds (stores) the value.

Why No Clock?

Because latches respond to the signal level directly, meaning:

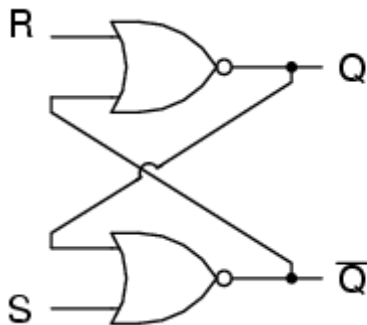
- As long as enable is HIGH → latch works
- When enable is LOW → latch stops

So, a clock edge is not required.

16.2 SR Latch

The SR Latch is the simplest bistable latch with S (Set) and R (Reset) inputs.

- Set → output becomes 1
- Reset → output becomes 0
- No change when both inputs are 0
- Invalid state when both inputs are 1 (for NOR latch)

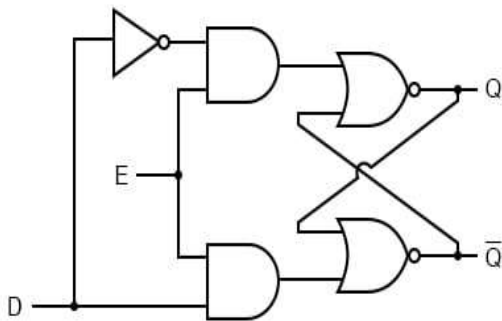


S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	0	0

16.3 D Latch

The D Latch eliminates the invalid condition of the SR latch by using a single D (Data) input. The output Q follows D when Enable (EN) is HIGH.

- When EN = 1, Q = D
- When EN = 0, Q holds previous value



E	D	Q	$\overline{Q}$
0	0	latch	latch
0	1	latch	latch
1	0	0	1
1	1	1	0

16.3 Latch VS Flip Flop

Feature	Latch	Flip-Flop
Triggering	Level-triggered	Edge-triggered
Clock Dependence	Works when enable/clock level is HIGH or LOW	Works only on rising/falling clock edge
Stability	Less stable, prone to glitches	Very stable, clean timing
Timing Control	Difficult to control	Easy to control
Race Condition	More chances (especially SR/JK latch)	Almost no race condition
Usage	Small circuits, buffers, synchronizers	Registers, counters, CPUs, SoCs, memory
Speed	Faster but uncontrolled	Slower but fully controlled
Design Complexity	Simple	More complex
Predictability	Low (due to level sensitivity)	High (due to edge triggering)
Setup & Hold Time	Often violated	Strictly maintained
Best For	Asynchronous circuits	Synchronous digital design

We avoid latches because they are level-triggered and can cause timing glitches and race conditions. Flip-flops are edge-triggered, stable, and ensure predictable timing required for synchronous digital design

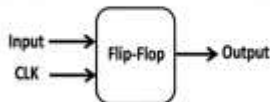
CHAPTER 17 – FLIP-FLOPS

17.1 Introduction

A **Flip-Flop** is a fundamental **bistable memory element** used in sequential digital circuits. It can **store one bit (0 or 1)** of information and changes its output only at a specific **clock edge** (rising or falling). Unlike **latches**, which are **level-triggered**, flip-flops are **edge-triggered**, making timing more reliable and predictable in **synchronous circuits**, such as **processors, memories, counters, and registers**.

Flip-flops are constructed using **logic gates** or **latches** and are the building blocks of **registers, shift registers, counters, and state machines**.

Flip-Flop



Clock Signal

Rising Edge



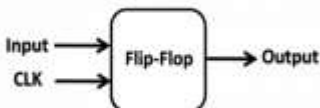
Positive Edge Triggered

Falling Edge

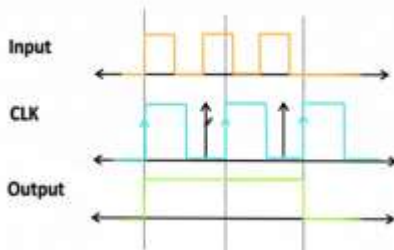


Negative Edge Triggered

Flip-Flop



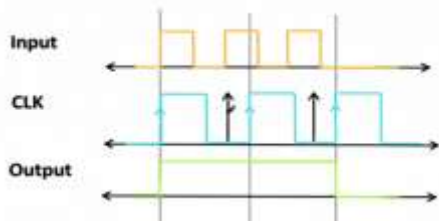
Positive Edge Triggered Flip-Flop



Flip-Flop



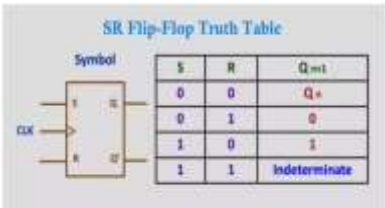
Positive Edge Triggered Flip-Flop



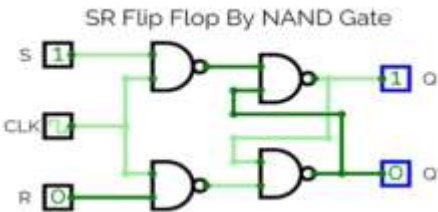
17.2 SR Flip-Flop

- ✓ **S = 0, R = 0 (Hold State):**When both S and R are 0, the latch remains in its previous state. It holds its current value.
- ✓ **S = 0, R = 1 (Reset State):**The latch is reset, meaning the output Q is forced to 0, and $\bar{Q}$ becomes 1.
- ✓ **S = 1, R = 0 (Set State):**The latch is set, which means the output Q is set to 1, and $\bar{Q}$ becomes 0.
- ✓ **S = 1, R = 1 (Invalid State):**Both S and R being 1 results in an invalid state where both Q and $\bar{Q}$ are undefined.

In practical circuits, this condition is avoided because it leads to unpredictable behavior.



Trigger	Inputs		Output				State
			Present State		Next State		
CLK	S	R	Q	Q'	Q	Q'	
	x	x	-	-	-	-	No Change
	0	0	0	1	0	1	No Change
	0	1	0	1	0	1	No Change
	1	0	0	1	1	0	Set
	1	1	0	1	-	-	Indeterminate



SR Flip Flop Truth Table

Q _n	Q _{n+1}	S	R
0	0	0	0
0	1	0	1
1	0	1	0
1	1	0	0

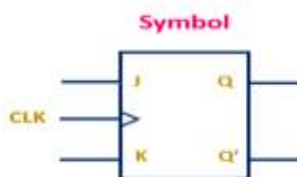
Q _n	S	R	Q _{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	X
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	X

Q _n	Q _{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Excitation Table

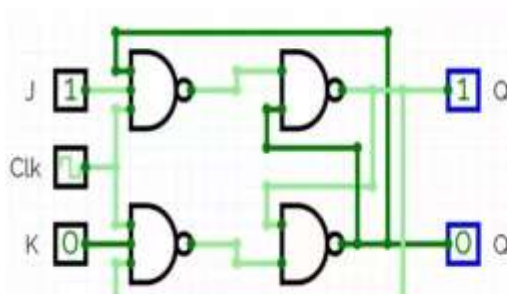
17.3 JK Flip-Flop

- **When J = 0 and K = 0:** The output (Q) remains the same as its previous state, maintaining the current state (**No change**).
- **When J = 0 and K = 1:** The output (Q) becomes 0, regardless of its previous state (**Reset**).
- **When J = 1 and K = 0:** The output (Q) becomes 1, regardless of its previous state (**Set**).
- **When J = 1 and K = 1:** The output (Q) toggles to its complement (**Complement / Toggle**).



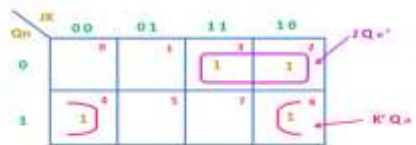
Truth Table

CLK	J	K	Q_{n+1}
↑	0	0	Q_n
↑	0	1	0
↑	1	0	1
↑	1	1	Q_n'



Trigger	Inputs		Output				State
			Present State		Next State		
CLK	J	K	Q	Q'	Q	Q'	
	x	x	-	-	-	-	Latched
	0	0	0	1	0	1	No Change
			1	0	1	0	
	0	1	0	1	0	1	Reset
			1	0	0	1	
	1	0	0	1	1	0	Set
			1	0	1	0	
	1	1	0	1	1	0	Toggle
			1	0	0	1	

	Q_n	J	K	Q_{n+1}
	0	0	0	0
m4	1	0	0	1
	0	0	1	0
	1	0	1	0
m2	0	1	0	1
m6	1	1	0	1
m3	0	1	1	1
	1	1	1	0



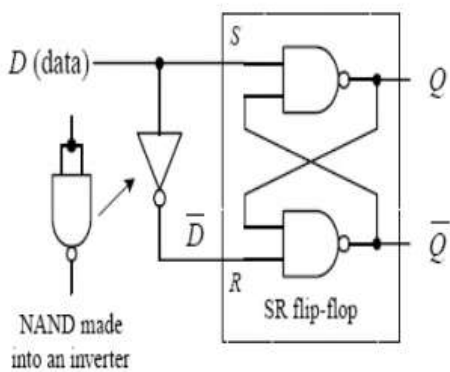
$$Q_{n+1} = JQ_n' + K'Q_n$$

Q_n	Q_{n+1}	J	K
0 → 0	0	0	X
1 → 0	0	X	1
0 → 1	1	1	X
1 → 1	1	X	0

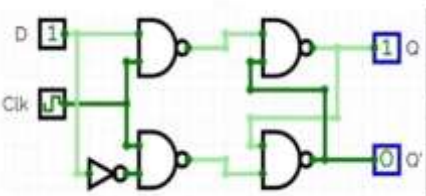
Excitation Table of JK Flip-Flop

17.4 D Flip-Flop

- If $D = 0$ at clock $\rightarrow Q$ becomes 0.
- If $D = 1$ at clock $\rightarrow Q$ becomes 1.
- Q will **not change** until the **next clock edge**.
- It works on **rising edge ($\uparrow$)** or **falling edge ($\downarrow$)** depending on design.
- Simple rule: **$Q(\text{next}) = D$** at the clock edge.



D	Q (Current)	Q (n+1) (Next)	Comment
0	0	0	reset
0	1	0	$Q(n+1) = 0$ always $D=0$ irrespective of the current state
1	0	1	When the input of the flip flop is 1, next state of flip flop will always be 1, regardless of the current state of the flip
1	1	1	reset



D	Q_n	Q_n'	Q_{n+1}
0	0	1	
0	1	0	
1	0	1	
1	1	0	

$Q(n+1) = D$

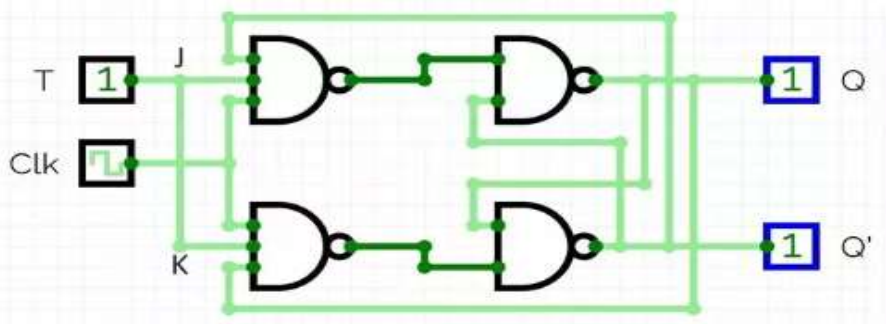
D Flip Flop Excitation Table

An excitation table is derived from the truth table.

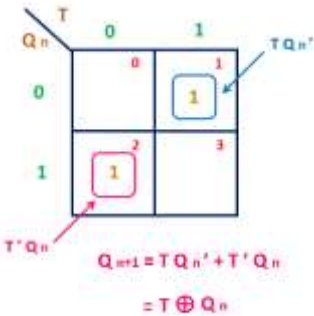
Q (Current)	D	Q (n+1) (Next)	Comment
0	0	0	hold state
0	1	1	reset state
1	0	0	hold state
1	1	1	reset

17.4 D Flip-Flop

- T Flip-Flop means "Toggle" Flip-Flop".
- T Flip-Flop means "Toggle" Flip-Flop".
- It changes (toggles) output Q when T = 1 at the clock edge.
- If T = 0, output Q stays the same the (no change).



CLK	T	Qn	Qn+1
	0	0	0
	0	1	1
	1	0	1
	1	1	0



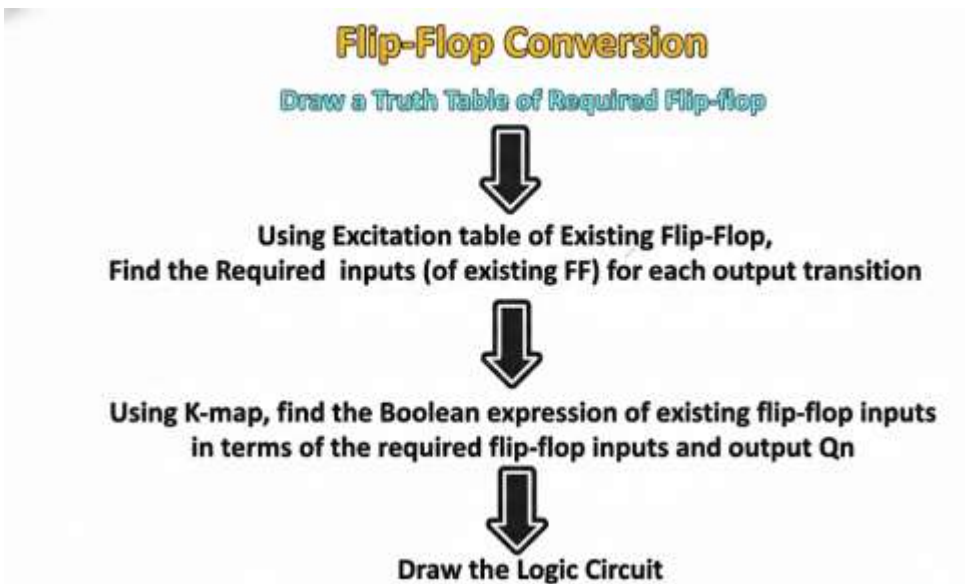
Qn	Qn+1	T
0 → 0	0	0
1 → 0	0	1
0 → 1	1	1
1 → 1	1	0

17.6 Flip-Flop Conversion

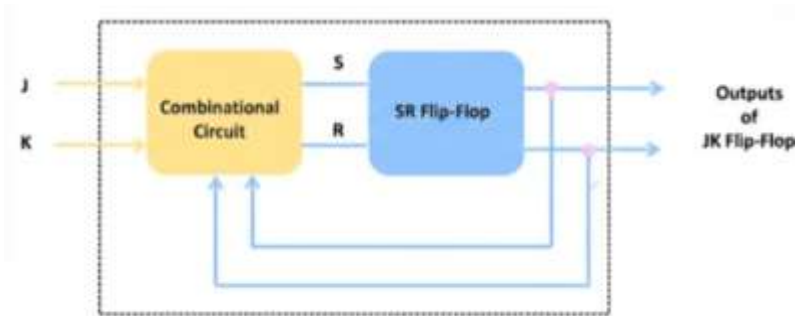
General Procedure for Flip-Flop Conversion

The conversion of flip-flops follows these steps:

1. Select Source Flip-Flop: This is the flip-flop you have (SR, JK, D, or T).
2. Select Target Flip-Flop: This is the flip-flop you need.
3. Write the Truth Table of Target Flip-Flop: Shows desired next state $Q(\text{next})$ based on inputs.
4. Write the Excitation Table of Source Flip-Flop: Shows what signals are required to get $Q(\text{next})$ from $Q(\text{previous})$.
5. Combine Tables: Use $Q(\text{previous})$, $Q(\text{next})$ to find the required source inputs.
6. Simplify Expressions: Use Boolean algebra or K-Map to get the minimal logic equation.
7. Draw the Logic Diagram: Connect the logic to the source flip-flop.



EXAMPLE : SR- JK FLIP-FLOP



J	K	Q_n	Q_{n+1}	S	R
0	0	0	0		
0	0	1	1		
0	1	0	0		
0	1	1	0		
1	0	0	1		
1	0	1	1		
1	1	0	1		
1	1	1	0		

Excitation Table of the SR Flip-Flop

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

J	K	Q_n	Q_{n+1}	S	R
0	0	1	0	0	X
0	1	1	0	X	0
0	1	1	0	0	X
0	1	1	0	0	1
1	0	1	0	X	1
1	0	1	0	X	0
1	1	0	1	1	0
1	1	1	0	0	1

CHAPTER 18 – REGISTERS AND COUNTERS

18.1 Shift Registers

A Shift Register is a sequential digital circuit used to store and shift data. It consists of a series of flip-flops connected in cascade, where the output of one flip-flop becomes the input of the next. With every clock pulse, data is shifted left or right.

Shift registers are used for:

- ✓ Serial to parallel conversion
- ✓ Parallel to serial conversion
- ✓ Data storage
- ✓ Delay circuits
- ✓ Counters
- ✓ Communication systems
- ✓ Digital signal processing

Basic Structure

A shift register with n flip-flops can store n bits of data. The shift direction is controlled by the logic design:

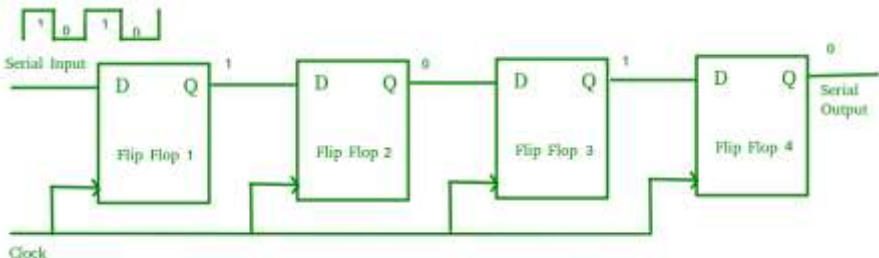
- Right Shift: Q output passes to the right
- Left Shift: Q output passes to the left

18.2 Types Of Register

Shift registers are classified based on input/output operation:

1. SISO – Serial In Serial Out

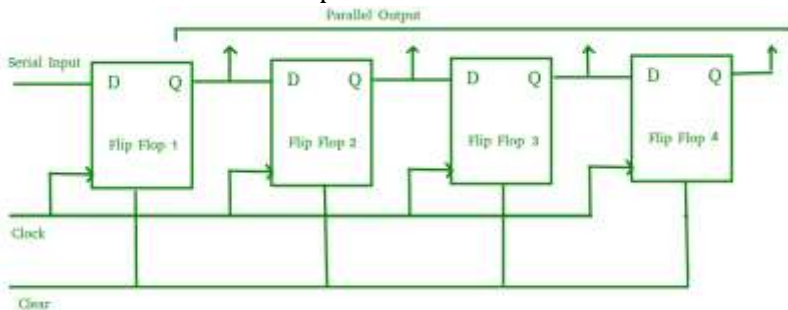
- ✓ Data is entered serially (bit-by-bit).
- ✓ Data is output serially.
- ✓ Used in simple data delay applications.



Operation: Input bit → Flip-Flop → Flip-Flop → ... → Output bit

2. SIPO – Serial In Parallel Out

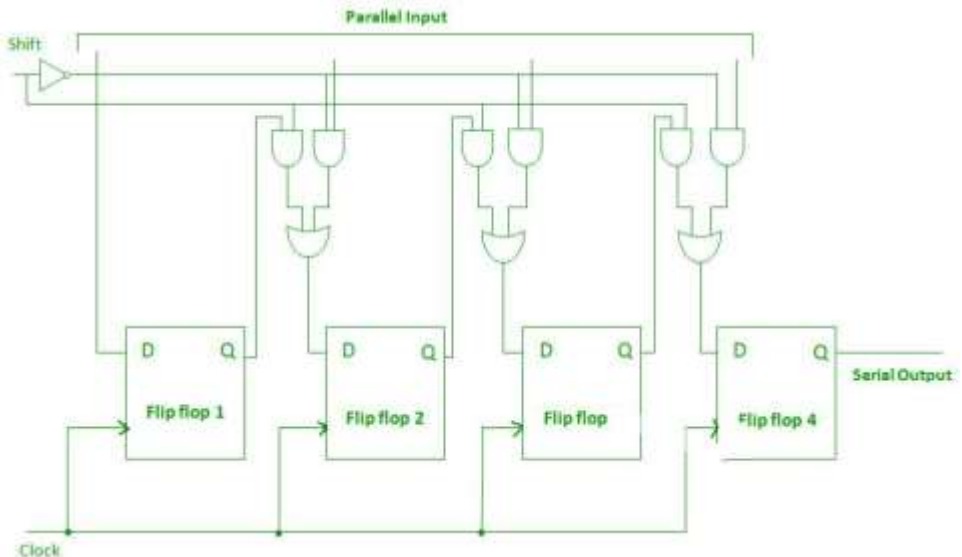
- ✓ Data is entered serially.
- ✓ Data is available in parallel after shifting.
- ✓ Used for serial-to-parallel conversion.



Example: Receiving serial data from communication line and converting to parallel for processing.

3. PISO – Parallel In Serial Out

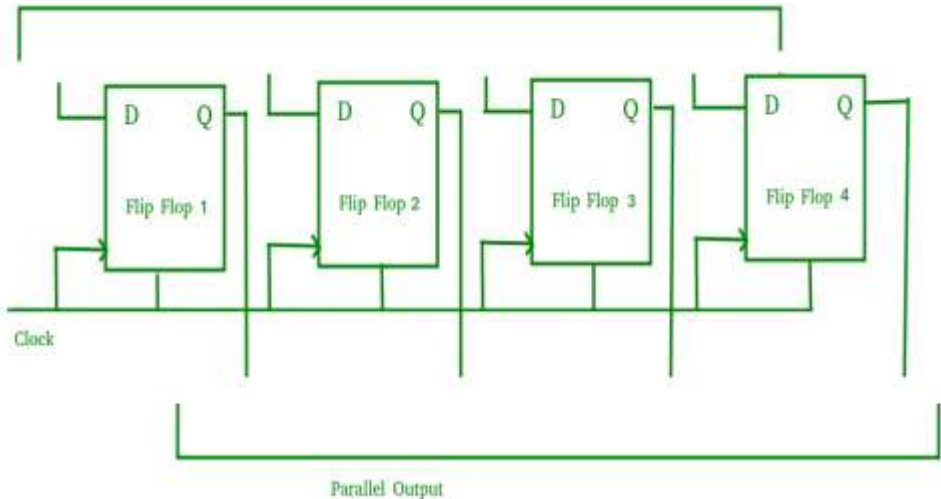
- ✓ Data is loaded in parallel to all flip-flops simultaneously.
- ✓ Data is shifted out serially.
- ✓ Used for parallel-to-serial conversion.



Example: Sending 8-bit data through a single communication line.

4. PIPO – Parallel In Parallel Out

- Data is loaded in parallel.
- Data is available in parallel.
- Used for temporary data storage.



Example: Registers inside CPU to store intermediate data.

18.3 Asynchronous Counters

An Asynchronous Counter, also known as a Ripple Counter, is a counter where the clock is applied only to the first flip-flop. The output of one flip flop acts as the clock input to the next flip-flop.

Because of this ripple effect, the flip-flops do not change state simultaneously. Hence, propagation delay accumulates in asynchronous counters.

Characteristics

- Simple design
- Low hardware requirement
- Slower due to ripple delay
- Used in low-speed applications

Applications

- Frequency division
- Divide-by-N circuits
- Simple event counters

18.4 Synchronous Counters

A Synchronous Counter is a digital counter where all flip-flops receive the clock simultaneously. The next state is generated using combinational logic connected to the flip-flops.

This makes synchronous counters fast, with no ripple delay, and suitable for high-speed digital systems.

Characteristics

- All flip-flops clocked together
- Higher speed than asynchronous
- Predictable timing
- Complex logic design

Applications

- Digital frequency counters
- Timers
- High-speed event counters
- Microprocessor clocking

18.5 Ring & Johnson Counters

(A) Ring Counter

A Ring Counter is a special type of shift register counter where the Q output of the last flip-flop is fed back to the input of the first flip-flop.

A ring counter with n flip-flops produces n unique states, typically circulating a single '1' bit around the ring.

Example for 4 flip-flops: 1000 $\rightarrow$ 0100 $\rightarrow$ 0010 $\rightarrow$ 0001 $\rightarrow$ 1000 ...

Applications

- Sequence generation
- LED chaser circuits
- Timing signals in ASIC/FPGA

(B) Johnson Counter (Twisted Ring Counter)

The Johnson Counter is also called a Modified Ring Counter. It connects the inverted output ($\bar{Q}$) of the last flip-flop to the input of the first flip-flop.

A Johnson counter with n flip-flops produces $2n$ states.

Example for 4 flip-flops:

0000 $\rightarrow$ 1000 $\rightarrow$ 1100 $\rightarrow$ 1110 $\rightarrow$ 1111 $\rightarrow$ 0111 $\rightarrow$ 0011 $\rightarrow$ 0001 $\rightarrow$ 0000 ...

CHAPTER 19 – FINITE STATE MACHINES (FSM)

A **Finite State Machine (FSM)** is a sequential digital logic model used to describe the behavior of a system in terms of **states**, **inputs**, and **outputs**. An FSM moves from one state to another based on **input conditions** and its **current state**. It is used widely in **digital design, communication protocols, controllers, and embedded systems**.

The two most common types of FSM are:

- Mealy Machine
- Moore Machine

19.1 Mealy Machine

A Mealy Machine is a finite state machine where the output depends on both the present state and the current input.

Characteristics

- $\text{Output} = f(\text{State}, \text{Input})$
- Output changes immediately when input changes
- Typically requires fewer states than Moore machine
- Faster response to input variations

Structure

A Mealy machine has:

- Finite set of states
- Input alphabet
- Output alphabet
- Next state function
- Output function based on state and input

Advantages

- Requires fewer states
- Faster because output depends directly on input

Disadvantages

- Output may be unstable due to immediate response
- More difficult to design and test

19.2 Moore Machine

A Moore Machine is a finite state machine where the output depends only on the present state, independent of the input.

Characteristics

- Output = $f(\text{State})$
- Output changes only when state changes
- More stable output
- Easier to design and debug

Structure

A Moore machine has:

- Finite set of states
- Input alphabet
- Output alphabet
- Next state function
- Output function based on state alone

Advantages

- Stable and predictable output
- Easier to verify and simulate

Disadvantages

- Usually requires more states than a Mealy machine
- Slightly slower response, since output changes only on next state